Gesture Mouse and Air Keyboard: Robust Real-Time Computer Control with a Webcam using Invariant Hand-Keypoint Learning

Guna Sekhar Subbarao Karanam
230537
kgssekhar23@iitk.ac.in

VOOBA RAJKUMAR
231173
rajvk23@iitk.ac.in

GBHUVANA REDDY
230408
bhuvanar23@iitk.ac.in

EE 604: Image Processing, IIT Kanpur, 2026

Abstract

Can a normal laptop webcam replace the mouse and the keyboard? We built a system that watches the user's hand and turns hand gestures into real mouse and keyboard actions: moving the cursor, left and right click, scrolling, dragging and typing on an on-screen "air keyboard". It needs no gloves, no special camera and no GPU. Simple versions of this idea already exist, but they decide the gesture with fixed "which fingers are up" rules, so they fail for new people and tilted hands, the cursor shakes, and it jumps away from the target at the moment of a click. We fixed these problems one by one. We first clean the 21 hand points found by MediaPipe so that position, size, tilt and left/right hand no longer matter, and a small neural network then learns the gestures from these cleaned points. A voting step over the last five frames stops false clicks. The cursor follows the index-finger knuckle instead of the fingertip, and a speed-aware One-Euro filter keeps it steady. On a person the network had never seen, it recognised gestures with ***96.6%*** *accuracy, against* ***89.8%*** *for the usual rules. The filter reduced cursor shake by* ***38%*** *with only* ***11 ms*** *of delay. Voting turned 50 detected clicks into exactly 30 for 30 real pinches. The knuckle cut the cursor jump during a click from 183 px to 53 px. The whole system runs at* ***32 frames per second*** *on a laptop CPU.*

1. Introduction

We touch a mouse and a keyboard all day. Sometimes touching them is a problem: a doctor wearing gloves, a cook with dirty hands, a teacher standing far from the laptop during a lecture, a shared screen in a public place, or a person who finds small devices hard to hold. Almost every laptop already has a webcam, so a mouse and keyboard that work with *only the webcam* would cost nothing and help all these people.

The idea is not new. Many open-source "virtual mouse" programs find the hand with MediaPipe [19], check which fingers are up with a few fixed rules, and move the cursor with the index fingertip. When we tried this approach, it worked in a demo but was hard to use for real work. We saw four problems:

1. **Rules break easily.** A slightly tilted hand, a hand far from the camera, or simply a different person makes the fixed thresholds fail.
2. **The cursor shakes.** The detected hand points move a few pixels in every frame even when the hand is still, so small buttons are hard to hit.
3. **The cursor jumps when you click.** To click, you pinch the thumb and index finger. This moves the fingertip, and the cursor goes with it, just when you need it to stay still.
4. **False clicks.** When the hand changes from one gesture to another, the detector is wrong for a frame or two, and each wrong "pinch" frame becomes a click.

Our project is about solving these four problems, and about *measuring* that each fix really helps. We keep MediaPipe for finding the hand, and everything after it is our own work:

- **A learned gesture classifier on cleaned hand points (Secs. 3.2 and 3.3).** Before classification we remove the effect of hand position, size, tilt and left/right hand. A small neural network (an MLP with about 20 k weights) then learns 7 gestures. We test it on a person it has never seen.
- **A stable cursor point (Sec. 3.5).** The cursor follows the index-finger *knuckle*, which hardly moves during a pinch.
- **A smart smoothing filter (Sec. 3.5).** The One-Euro filter [2] smooths strongly when the hand is slow and weakly when it is fast, so we get a steady cursor without delay.
- **Voting over time (Sec. 3.4).** A gesture counts only when it is seen in most of the last five frames with good confidence.
- **Extras that make it usable (Sec. 3.6).** An air keyboard

with a "hover-lock" that types the right key, and a one-second personalization for a new user.

The rest of the report follows the same story. Section 2 looks at what others have done. Section 3 explains our method step by step, and why we chose each step. Section 4 tests every step with our own recorded data and live sessions. Section 5 sums up what we learned.

2. Related Work

Recognising gestures from pixels. Early systems found the hand by its skin colour or by removing the background, and then described its shape; Rautaray and Agrawal give a survey [12]. These methods break easily when the light or the background changes. Deep learning changed the field. 3D CNNs with recurrent layers recognise moving gestures in video [10], and two-stage CNN systems detect and classify gestures in real time [7]. Large datasets such as Jester [9] and HaGRID [6] made this possible. These models work very well, but they read raw pixels, so they need a lot of training data and usually a GPU.

Finding hand points. Another line of work finds the joints of the hand instead of classifying pixels. Simon *et al*. [13] detect 21 hand keypoints in a single image, and MediaPipe Hands [19] does this in real time on a phone CPU. Once we have these 21 points, the background, skin colour and lighting are already gone, and the gesture problem becomes much smaller: only 63 numbers per frame. This is why we build on MediaPipe.

Virtual mouse programs. Many open-source projects join MediaPipe with PyAutoGUI [16]. They count fingers as "up" or "down" with fixed thresholds and move the cursor with the fingertip. They are easy to build, but, as said above, they are fragile, shaky and click by mistake. They are also rarely measured: they show a demo, not numbers.

Smoothing and measuring a pointer. Shaky input is usually smoothed with a moving average or a Kalman filter [5]. Both trade shake for delay. The One-Euro filter [2] changes its smoothing with the speed of the hand, which removes most of this trade-off. Pointing devices are usually compared with Fitts' law [3, 14].

Where our work fits. We take the keypoint idea from the second line of work and the virtual-mouse goal from the third, and we add what is missing in between: a learned and position-free classifier tested on new users, and a cursor that is designed and measured to be steady and reliable.

3. Proposed Method

Figure 1 shows the whole journey of one camera frame. In this section we follow that frame from left to right. For each step we say what it does and why we need it.

3.1. Step 1: finding the hand

The webcam gives a 960×540 colour frame about 30 times per second. We flip it like a mirror, so that moving the hand to the right moves the cursor to the right. MediaPipe's Hand Landmarker [19] then returns 21 points $\mathbf{p}_i = (x_i, y_i, z_i)$ on the hand and tells us whether it is a left or a right hand. Point 0 is the wrist, point 4 the thumb tip, point 5 the index knuckle and point 8 the index tip. *Why:* MediaPipe is accurate, runs on a CPU and tracks the hand between frames, so most of our time budget is left for the next steps.

3.2. Step 2: cleaning the hand points

The same gesture can give very different raw numbers. The hand may be on the left or the right of the image, near or far from the camera, tilted, or it may be a left hand. A classifier would have to learn all these cases from data, and our dataset is small. So we remove them by geometry (Fig. 2):

1. **Mirror** a left hand, $x_i \leftarrow 1 - x_i$, so that one model works for both hands.
2. **Move** the wrist to the origin: $\mathbf{q}_i = \mathbf{p}_i - \mathbf{p}_0$.
3. **Scale** by the palm size, the distance from the wrist to the middle-finger knuckle: $\mathbf{q}_i \leftarrow \mathbf{q}_i / \|(\mathbf{p}_9 - \mathbf{p}_0)_{xy}\|$.
4. **Rotate** the points so that the palm always points straight up.

After this, a pointing hand looks the same wherever it is in the image. We then add 15 bend angles (three joints per finger) and the 10 distances between every pair of fingertips. These describe the hand shape directly: a pinch is simply a small thumb-to-index distance. This gives $63{+}15{+}10 = 88$ numbers per frame.

3.3. Step 3: naming the gesture with a neural network

We use 7 gestures (Fig. 3). A small multi-layer perceptron (MLP) reads the 88 features and outputs a probability for each gesture. Its layers are $88 \to 128 \to 64 \to 7$, with batch normalisation [4], ReLU and dropout [15]. It has about 20 k weights. To make it work for people who are not in the training data, we show it four extra copies of every training frame. Each copy is slightly rotated (up to $\pm 20°$), scaled (0.85 to 1.15), shifted and given a little noise (data augmentation).

Why an MLP and not a CNN on the image? After Step 2 the input is only 88 numbers, so a tiny network is enough. It runs in under a millisecond on a CPU, and it never sees the background or the lighting. We check this choice against an image CNN in Sec. 4.2.

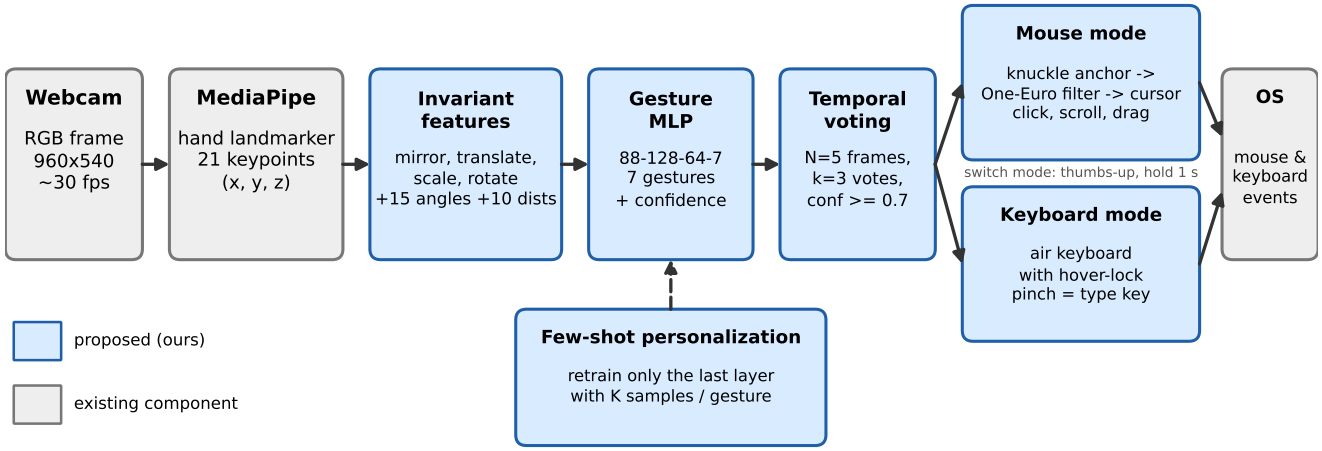


Figure 1. **Overview of our system.** Grey boxes are existing tools and blue boxes are our work. Each webcam frame gives 21 hand points. We clean them into 88 features, the Gesture MLP names the gesture, and voting makes the decision stable. The stable gesture then drives either the mouse (knuckle anchor and One-Euro filter) or the air keyboard (hover-lock). A new user can personalise the MLP with a few samples.

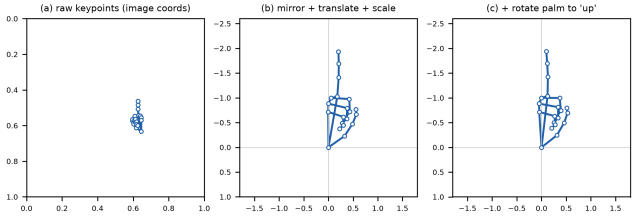


Figure 2. **Cleaning the hand points** for one recorded frame: raw points in the image (a), after mirroring, moving and scaling (b), and after rotating the palm to point up (c).

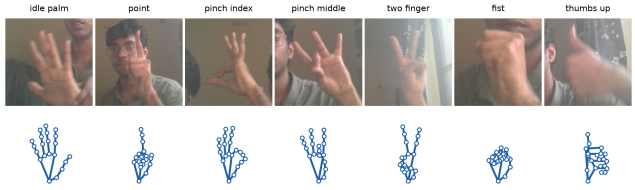


Figure 3. **Our 7 gestures.** Top: a frame from our dataset. Bottom: the typical cleaned hand points of that gesture.

3.4. Step 4: voting over time

When the hand moves from "point" to "pinch", the network can be unsure for a frame or two. If we acted on every frame, these short mistakes would become clicks. So we keep the last $N = 5$ predictions and their confidences, and we change the *stable* gesture only when one gesture gets at least $k = 3$ of the 5 votes *and* its average confidence is at least 0.7. Otherwise the old gesture stays. Actions happen only at the moment a stable gesture *starts*, with at least 0.35 s between two clicks. The cost is a delay of 2–3 frames, which is below 100 ms.

3.5. Step 5: a steady cursor

Which point should the cursor follow? Most virtual mice use the index fingertip. But a click is a pinch, and a pinch moves the fingertip towards the thumb. Because of the voting delay, the cursor follows that moving fingertip for a few frames before the click happens, so the click lands next to the target. We use the index **knuckle** (point 5) instead. It sits under the pointing finger, so pointing still feels natural, but it hardly moves during a pinch (Fig. 4).

From hand to screen. A box in the middle of the camera view (the "active area") is stretched to fill the whole screen. A small hand movement can therefore reach every corner.

Smoothing with the One-Euro filter. Even a still hand gives slightly different points in every frame. A plain moving average removes this shake but makes the cursor lag behind the hand. The One-Euro filter [2] is a low-pass filter whose cut-off frequency grows with the hand's speed:

$$\hat{X}_t = \alpha_t X_t + (1-\alpha_t)\,\hat{X}_{t-1}, \quad \alpha_t = \frac{1}{1+\frac{1}{2\pi f_c\,\Delta t}}, \quad f_c = f_{\min} + \beta\,|\dot{X}_t|. \tag{1}$$

When the hand is slow, f_c is low and the shake is removed. When the hand is fast, f_c is high and there is no lag. We use $f_{\min} = 1$ Hz and $\beta = 0.02$ on screen pixels.

3.6. Step 6: actions and the air keyboard

Table 1 lists what each stable gesture does. Actions are sent to the operating system with PyAutoGUI [16]. Holding a

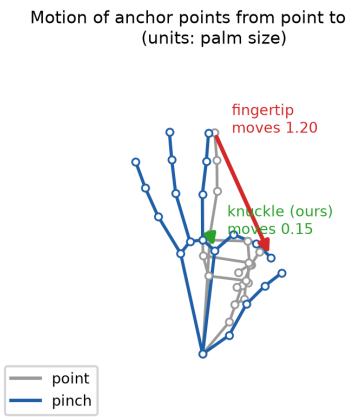


Figure 4. **Why we use the knuckle.** A typical "point" hand (grey) and "pinch" hand (blue) from our data, placed on the same wrist. When the user pinches, the fingertip moves much more than the knuckle.

Table 1. Gestures and what they do.

Gesture	Mouse mode	Keyboard mode
Open palm	pause (cursor stays)	–
Point (index up)	move cursor	highlight key
Pinch thumb + index	left / double click	type key
Pinch thumb + middle	right click	–
Two fingers up (V)	scroll	–
Fist	drag	–
Thumbs up, 1 s	to keyboard mode	to mouse mode

thumbs-up for one second switches to keyboard mode. A QWERTY keyboard is then drawn on the camera view, and the key under the index fingertip is highlighted. A pinch types it. Here the pinch again moves the fingertip, by about one key, so we add a **hover-lock**: we type the key that was highlighted in the last "point" frame *before* the pinch.

3.7. Step 7: one-second personalization

Every person makes gestures a little differently. Early layers of a network learn general features, and the last layer is the most task-specific one [18]. So for a new user we freeze the network and re-train only its last layer (455 weights) on K samples per gesture from that user. This takes about one second on a CPU and can be done just before use.

4. Experiments

We now test the method step by step. The questions follow the four problems of Sec. 1: does the classifier work on new people, does the filter stop the shake, does voting stop false clicks, and does the knuckle stop the jump?

Table 2. Gesture classification on the random 20% test split (accuracy and macro-F1 in %, latency per sample on one CPU thread).

Method	Acc.	F1	ms	Params
Rule-based (fingers up)	89.89	90.04	0.033	–
SVM (RBF), our features	99.74	99.74	0.150	–
Random Forest, our features	**99.84**	**99.84**	10.871	–
kNN (k=5), our features	99.77	99.78	1.630	–
MLP, raw coordinates	99.67	99.67	0.222	17.3k
MLP, + mirror/translate/scale	99.80	99.80	0.293	17.3k
MLP, + rotation	99.80	99.80	0.317	17.3k
MLP, full features, no augm.	99.77	99.78	0.776	20.5k
MLP, full features + augm. (ours)	99.74	99.74	0.720	20.5k

4.1. Dataset and setup

Our dataset. No public dataset has our 7 control gestures together with hand points, so we recorded our own with our tool `collect_data.py`. Each of the three group members recorded every gesture while moving the hand around the image, closer and farther, and a little tilted. In total we have **15,378 labelled frames** (4,774 to 5,367 per person and 1,880 to 2,530 per gesture), each with its 21 hand points and handedness, plus **7,692 hand-crop images** for the CNN baseline.

Two kinds of test. In the *random split*, 20% of all frames are kept for testing. The same people appear in training and testing, so this test is easy. In *leave-one-user-out*, we train on two people and test on the third, for each person in turn. This is the honest test, because it is exactly what happens when a new person uses the system.

Live sessions. For the cursor experiments we ran the app and saved every frame's raw hand points and predictions to a log. We recorded three one-minute sessions: holding the hand still, moving the cursor around, and pinching exactly 30 times. Because the raw data is saved, we can replay the *same* hand motion through every filter or voting setting and compare them fairly.

Implementation. Python with OpenCV [1], MediaPipe, PyTorch [11], scikit-learn and PyAutoGUI [16]. The MLP is trained with AdamW [8] (learning rate 10^{-3}, cosine schedule, label smoothing 0.05 [17]) for 60 epochs with batch size 64. We keep the epoch with the best validation accuracy. Everything runs on an ordinary laptop CPU without a GPU; timings are for one frame on one CPU thread.

4.2. Can it recognise the gestures?

Table 2 shows the random split. The usual rule-based method gets 89.9%. Every learned method gets between 99.7% and 99.8%, and our MLP takes only 0.72 ms per frame. On this easy test the learned methods are all about

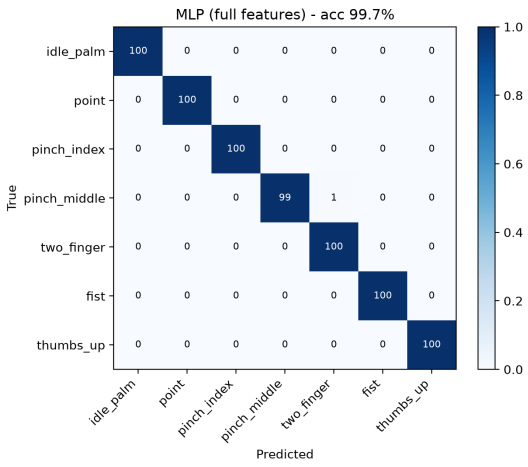


Figure 5. Confusion matrix (%) of our MLP on the random test split.

Table 3. Leave-one-user-out accuracy (%): trained on the other users, tested on the user in the row.

Test user	Rules	SVM	MLP raw	MLP ours
bhuvana	85.15	76.50	91.24	94.39
guna	93.50	99.05	95.73	98.30
raj	90.85	95.04	94.65	97.23
Mean	**89.83**	**90.19**	**93.88**	**96.64**

equally good, and the confusion matrix (Fig. 5) shows almost no mistakes. The cleaning steps and augmentation change little here, because the test frames come from the same people as the training frames.

The real test: a new person. Table 3 tells a different story. When the person is new, the rules drop to 89.8% and the SVM drops to 90.2%, with only 76.5% for Bhuvana's hand. An MLP on the raw points without augmentation reaches 93.9%. Our cleaned features with augmentation reach **96.6%** on average, and are the best for two of the three people. So the cleaning and augmentation pay off exactly where they are needed: on hands the model has not seen.

Hand points or image pixels? Table 4 compares our MLP with a CNN trained on the hand-crop images of the same frames. The CNN is slightly more accurate on the random split (99.94% vs. 99.74%). However, it is about 5.5 times slower (4.0 ms vs. 0.72 ms per frame), 12 times bigger (242 k vs. 20 k weights), and took 455 s to train. For a system that must leave time for tracking and control in every frame, the small keypoint MLP is the better choice.

Table 4. Classifier on image pixels vs. on keypoints (same frames, random split).

Input / model	Acc.	F1	ms	Params
CNN, 64×64 hand image	**99.94**	**99.94**	4.01	242k
MLP, keypoint features (ours)	99.74	99.74	0.72	20.5k

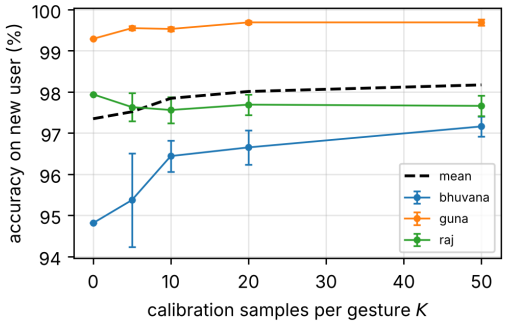


Figure 6. Accuracy on a new user against the number of calibration samples per gesture K ($K = 0$ means no calibration).

Table 5. Cursor smoothing on the same recorded hand motion (lower is better for all columns).

Filter	Jitter (px/frame)	Error moving (px)	Lag (ms)
No filter (raw)	4.21	**15.19**	**0.00**
Moving average (5)	3.57	61.52	64.03
Kalman (const. velocity)	3.66	37.64	32.02
One-Euro (ours)	**2.59**	18.82	10.65

Does personalization help? Figure 6 shows the accuracy on a new user when only the last layer is re-trained with K samples per gesture. Averaged over the three of us it rises from 97.4% ($K = 0$) to 98.0% ($K = 20$). The gain is largest for the hardest user, Bhuvana: from 94.8% to 96.7% with 20 samples and 97.2% with 50. For the other two, who already started near 98–99%, it changes little.

4.3. Is the cursor steady?

We replayed the recorded hand motion through four filters (Tab. 5 and Fig. 7). *Shake* is how far the cursor moves per frame while the hand is held still. *Lag* is how late the filtered cursor is compared with the intended path.

Without a filter the cursor shakes by 4.21 px every frame. The moving average reduces this only to 3.57 px and adds 64 ms of lag, so the cursor feels like it is pulled on a rubber band. The Kalman filter is in between. The One-Euro filter gives the **least shake (2.59 px, 38% less than no filter)** with only **11 ms** of lag. Its error while moving (18.8 px) is also close to the raw cursor's, while the moving average is far behind (61.5 px).

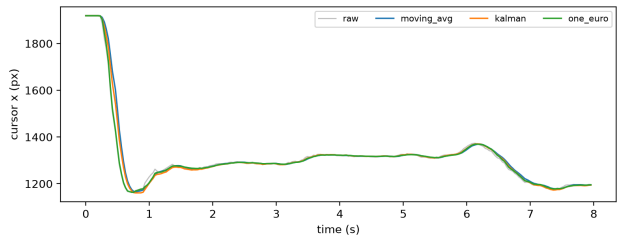


Figure 7. Cursor x-position over time in a recorded session: raw and filtered.

Table 6. Effect of temporal voting on the same recorded predictions (the user clicked 30 times).

Setting	Changes/min	Short false/min	Clicks
No voting (raw per-frame)	70.0	31.0	50
Voting N=3, k=2	24.9	1.2	32
Voting N=5, k=3 (ours)	21.2	0.3	30
Voting N=7, k=5	18.1	0.0	28

Table 7. Cursor drift at the moment of a click for different anchor points (screen pixels, 1920×1080).

Anchor	Mean (px)	Median (px)	Clicks
Index fingertip (usual)	182.5	161.1	30
Index knuckle (ours)	53.4	51.5	30
Palm centre	**52.1**	**48.9**	30

4.4. Does voting stop false clicks?

In the third session the user pinched exactly 30 times. We replayed the network's frame-by-frame predictions with and without voting (Tab. 6).

Without voting, the gesture changed 70 times per minute, including 31 short false changes of one or two frames per minute, and **50 clicks** were detected for 30 real pinches. With our setting ($N = 5$, $k = 3$) the short false changes almost vanish (0.3 per minute) and **exactly 30 clicks** are detected. A stricter setting ($N = 7$, $k = 5$) starts to miss real clicks (28), so $N = 5$, $k = 3$ is the right balance.

4.5. Does the knuckle stop the jump?

For each of the 30 clicks we measured how far the cursor would move in the four frames before the click is recognised, for three choices of anchor point (Tab. 7).

With the fingertip, the cursor moves **183 px** on average during a click, which is enough to miss most buttons. With the knuckle it moves only **53 px**, about 3.4 times less. The palm centre is about as stable (52 px), but it sits away from the pointing finger and moves when the wrist bends, so pointing with it feels less natural. We therefore keep the knuckle.

Table 8. System performance and user tests.

Metric	Value
Frame rate (FPS)	32.2
Processing per frame, mean (ms)	11.5
Processing per frame, 95th perc. (ms)	14.8

4.6. Is it fast enough?

Over 7,570 logged frames the complete system ran at **32.2 frames per second**, with 11.5 ms of processing per frame on average (14.8 ms in the slowest 5% of frames). It works in real time on an ordinary laptop CPU (Tab. 8).

4.7. Limitations

The system follows one hand at a time, and the hand must stay inside the active area. Dark rooms and fast motion blur make MediaPipe lose the hand. Typing on the air keyboard is much slower than on a real keyboard, so it is meant for short text such as a search word. Finally, our data and tests come from only three people.

5. Conclusion

We started with a simple question: can a webcam replace the mouse and the keyboard? Existing virtual mice showed that it is possible, but also that they are fragile, shaky and click by mistake. We went after each of these problems and measured every fix. Cleaning the hand points and learning the gestures with a small network raised accuracy on a new person from 89.8% to 96.6%. The One-Euro filter cut the cursor shake by 38% with only 11 ms of delay. Voting turned 50 detected clicks into exactly the 30 the user made, and the knuckle anchor cut the cursor jump during a click from 183 px to 53 px. The system runs at 32 frames per second on a laptop CPU, and a new user can personalise it in about a second. In future we would like to add moving gestures such as swipes, a relative "lift and place" cursor mode, and tests with many more people.

Individual contributions

- **Guna Sekhar Subbarao Karanam (230537):** Gesture recognition: data-collection tool, invariant feature design, Gesture MLP training with augmentation, ablation study, baselines (rules, SVM, RF, kNN, image CNN) and leave-one-user-out evaluation.
- **VOOBA RAJKUMAR (231173):** Cursor control: stable anchor point, One-Euro / Kalman / moving-average filters, confidence + temporal voting, click / scroll / drag logic, recording and analysis of the live cursor sessions.
- **GBHUVANA REDDY (230408):** User interface and air keyboard with hover-lock, few-shot personalization, figures, report and presentation.

References

[1] Gary Bradski. The OpenCV library. *Dr. Dobb's Journal of Software Tools*, 2000. 4

[2] Géry Casiez, Nicolas Roussel, and Daniel Vogel. 1 € filter: A simple speed-based low-pass filter for noisy input in interactive systems. In *ACM CHI Conference on Human Factors in Computing Systems*, 2012. 1, 2, 3

[3] Paul M. Fitts. The information capacity of the human motor system in controlling the amplitude of movement. *Journal of Experimental Psychology*, 47(6):381–391, 1954. 2

[4] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *ICML*, 2015. 2

[5] Rudolph E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1): 35–45, 1960. 2

[6] Alexander Kapitanov, Karina Kvanchiani, Alexander Nagaev, Roman Kraynov, and Andrei Makhliarchuk. HaGRID – HAnd Gesture Recognition Image Dataset. In *WACV*, 2024. 2, 8

[7] Okan Köpüklü, Ahmet Gunduz, Neslihan Kose, and Gerhard Rigoll. Real-time hand gesture detection and classification using convolutional neural networks. In *IEEE International Conference on Automatic Face & Gesture Recognition (FG)*, 2019. 2

[8] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019. 4

[9] Joanna Materzynska, Guillaume Berger, Ingo Bax, and Roland Memisevic. The Jester dataset: A large-scale video dataset of human gestures. In *ICCV Workshops*, 2019. 2

[10] Pavlo Molchanov, Xiaodong Yang, Shalini Gupta, Kihwan Kim, Stephen Tyree, and Jan Kautz. Online detection and classification of dynamic hand gestures with recurrent 3D convolutional neural networks. In *CVPR*, 2016. 2

[11] Adam Paszke et al. PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS*, 2019. 4

[12] Siddharth S. Rautaray and Anupam Agrawal. Vision based hand gesture recognition for human computer interaction: a survey. *Artificial Intelligence Review*, 43(1):1–54, 2015. 2

[13] Tomas Simon, Hanbyul Joo, Iain Matthews, and Yaser Sheikh. Hand keypoint detection in single images using multiview bootstrapping. In *CVPR*, 2017. 2

[14] R. William Soukoreff and I. Scott MacKenzie. Towards a standard for pointing device evaluation, perspectives on 27 years of Fitts' law research in HCI. *International Journal of Human-Computer Studies*, 61(6):751–789, 2004. 2

[15] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15:1929–1958, 2014. 2

[16] Al Sweigart. PyAutoGUI: cross-platform gui automation for python. https://github.com/asweigart/pyautogui, 2014. 2, 3, 4

[17] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *CVPR*, 2016. 4

[18] Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? In *NeurIPS*, 2014. 4

[19] Fan Zhang, Valentin Bazarevsky, Andrey Vakunov, Andrei Tkachenka, George Sung, Chuo-Ling Chang, and Matthias Grundmann. MediaPipe Hands: On-device real-time hand tracking. In *CVPR Workshop on Computer Vision for Augmented and Virtual Reality*, 2020. arXiv:2006.10214. 1, 2

A. Code, repository and datasets

Our GitHub repository: https://github.com/gunasekharkaranam/gesture-mouse-keyboard

Datasets used.

- Our own recorded dataset (15,378 labelled hand-point frames and 7,692 hand-crop images from 3 users, 7 gestures), in the repository under `data/`.
- Related public dataset (not used for training, listed for reference): HaGRID [6], https://github.com/hukenovs/hagrid.

Repositories and libraries used.

- MediaPipe (hand landmarks): https://github.com/google-ai-edge/mediapipe
- OpenCV (camera and drawing): https://github.com/opencv/opencv
- PyTorch (neural networks): https://github.com/pytorch/pytorch
- scikit-learn (SVM, Random Forest, kNN baselines): https://github.com/scikit-learn/scikit-learn
- PyAutoGUI (mouse and keyboard events): https://github.com/asweigart/pyautogui
- CVPR author kit (this template): https://github.com/cvpr-org/author-kit
- One-Euro filter reference page: https://gery.casiez.net/1euro/

How to run. `pip install -r requirements.txt`; record data with `python collect_data.py --user NAME`; run every experiment with `python run_all.py`; start the app with `python app.py`. Details are in `README.md`.

B. Complete source code

`app.py`

```
"""
app.py
------
Main application: control the mouse and keyboard with hand gestures.

    python app.py                        # normal use
    python app.py --dry-run              # show everything but do NOT move the real mouse
    python app.py --log                  # save a per-frame log in logs/ (for experiments)
    python app.py --model models/gesture_mlp_guna.pt   # use a personalized model

Gestures (MOUSE mode)
    point (index up)        move cursor
    pinch thumb+index       left click (pinch twice quickly = double click)
    pinch thumb+middle      right click
    two fingers up          scroll (move hand up / down)
    fist                    drag (hold left button while moving)
    open palm               pause (cursor stays still)
    thumbs up, hold 1 s     switch to KEYBOARD mode (and back)

KEYBOARD mode: point at a key on the air keyboard, pinch to type it.

Keys in the window
    q / ESC quit           c  control on/off      k  switch mode
    f  next filter         a  next anchor point   v  voting on/off
    r  model <-> rule-based classifier            l  logging on/off
"""
import argparse
import csv
import os
import time

import cv2
import numpy as np

import config
from controller import Controller
from filters import make_filter, FILTERS
from hand_tracker import HandTracker, draw_hand
from rules import classify_rules
from virtual_keyboard import VirtualKeyboard
from voting import GestureVoter

ANCHORS = ["index_mcp", "index_tip", "palm"]
G = {name: i for i, name in enumerate(config.GESTURES)}

```

```
46
47  def anchor_point(lm, anchor):
48      """The hand point that drives the cursor (normalized image coords)."""
49      if anchor == "index_tip":
50          return float(lm[8, 0]), float(lm[8, 1])
51      if anchor == "index_mcp":
52          return float(lm[5, 0]), float(lm[5, 1])
53      p = lm[[0, 5, 17]].mean(0)                      # palm centre
54      return float(p[0]), float(p[1])
55
56
57  def to_screen(x, y, screen_w, screen_h, region=config.ACTIVE_REGION):
58      """Map a point inside the active region of the camera frame to screen pixels."""
59      x0, y0, x1, y1 = region
60      u = np.clip((x - x0) / (x1 - x0), 0, 1)
61      v = np.clip((y - y0) / (y1 - y0), 0, 1)
62      return u * (screen_w - 1), v * (screen_h - 1)
63
64
65  class GestureEngine:
66      """
67      All the decision logic, separated from the camera so it can be tested.
68      step(hand, t) is called once per frame and returns a list of events.
69      """
70
71      def __init__(self, controller, predictor=None, filter_name=config.DEFAULT_FILTER,
72                   anchor=config.DEFAULT_ANCHOR, use_voting=True, frame_size=(960, 540)):
73          self.ctrl = controller
74          self.predictor = predictor                # None -> rule-based classifier
75          self.use_model = predictor is not None
76          self.filter_name = filter_name
77          self.filter = make_filter(filter_name)
78          self.tip_filter = make_filter("one_euro")  # for keyboard hover
79          self.anchor = anchor
80          self.voter = GestureVoter(enabled=use_voting)
81          self.keyboard = VirtualKeyboard(*frame_size)
82          self.frame_w, self.frame_h = frame_size
83          self.mode = "MOUSE"
84          self.control_on = True
85          self.prev_stable = G["idle_palm"]
86          self.last_click_t = 0.0
87          self.thumbs_start = None
88          self.mode_switched = False
89          self.scroll_prev_y = None
90          self.cursor = (controller.screen_w / 2, controller.screen_h / 2)
91          self.raw_label, self.conf, self.stable = G["idle_palm"], 0.0, G["idle_palm"]
92
93      # ------------------------------------------------------------------
94      def classify(self, hand):
95          if self.use_model and self.predictor is not None:
96              return self.predictor.predict(hand.landmarks, hand.handedness)
97          return classify_rules(hand.landmarks, hand.handedness)
98
99      def set_filter(self, name):
100         self.filter_name, self.filter = name, make_filter(name)
101
102     def _click_allowed(self, t):
103         return t - self.last_click_t >= config.CLICK_COOLDOWN_S
104
105     # ------------------------------------------------------------------
106     def step(self, hand, t):
107         events = []
108         if hand is None:                          # hand lost: release everything
109             if self.ctrl.mouse_is_down:
110                 self.ctrl.mouse_up(); events.append("drag_end")
111             self.voter.reset(); self.filter.reset(); self.tip_filter.reset()
112             self.prev_stable = self.stable = G["idle_palm"]
113             self.thumbs_start, self.scroll_prev_y = None, None
114             return events
115
116         self.raw_label, self.conf = self.classify(hand)
117         s = self.voter.update(self.raw_label, self.conf)
118         self.stable = s
119         onset = s != self.prev_stable             # gesture just started this frame
120         lm = hand.landmarks
121
122         # ----- mode switch: hold thumbs-up -----
123         if s == G["thumbs_up"]:
124             if self.thumbs_start is None:
```

```
                self.thumbs_start, self.mode_switched = t, False
            elif not self.mode_switched and t - self.thumbs_start >= config.MODE_SWITCH_HOLD_S:
                self.mode = "KEYBOARD" if self.mode == "MOUSE" else "MOUSE"
                self.mode_switched = True
                events.append(f"mode_{self.mode.lower()}")
        else:
            self.thumbs_start = None

        if self.mode == "MOUSE":
            events += self._mouse(s, onset, lm, t)
        else:
            events += self._keyboard(s, onset, lm, t)

        self.prev_stable = s
        return events

    # ------------------------------------------------------------------
    def _mouse(self, s, onset, lm, t):
        ev = []
        ax, ay = anchor_point(lm, self.anchor)
        sx, sy = to_screen(ax, ay, self.ctrl.screen_w, self.ctrl.screen_h)
        fx, fy = self.filter(sx, sy, t)          # smooth in screen pixels

        # cursor moves only while pointing or dragging; frozen otherwise
        if s in (G["point"], G["fist"]):
            self.cursor = (fx, fy)
            if self.control_on:
                self.ctrl.move(fx, fy)

        # drag start / end
        if s == G["fist"] and onset:
            if self.control_on: self.ctrl.mouse_down()
            ev.append("drag_start")
        if s != G["fist"] and self.ctrl.mouse_is_down:
            self.ctrl.mouse_up(); ev.append("drag_end")

        # clicks (on gesture onset, with cooldown)
        if onset and s == G["pinch_index"] and self._click_allowed(t):
            if self.control_on: self.ctrl.click("left")
            self.last_click_t = t; ev.append("left_click")
        if onset and s == G["pinch_middle"] and self._click_allowed(t):
            if self.control_on: self.ctrl.click("right")
            self.last_click_t = t; ev.append("right_click")

        # scroll: vertical hand motion while two fingers are up
        if s == G["two_finger"]:
            if self.scroll_prev_y is not None:
                dy = self.scroll_prev_y - ay              # hand up -> positive -> scroll up
                if abs(dy) > config.SCROLL_DEADZONE:
                    amount = int(dy * config.SCROLL_GAIN)
                    if self.control_on: self.ctrl.scroll(amount)
                    ev.append(f"scroll_{amount}")
            self.scroll_prev_y = ay
        else:
            self.scroll_prev_y = None
        return ev

    def _keyboard(self, s, onset, lm, t):
        ev = []
        # fingertip in frame pixels (smoothed) -> hovered key
        px, py = self.tip_filter(lm[8, 0] * self.frame_w, lm[8, 1] * self.frame_h, t)
        self.tip_px = (int(px), int(py))
        self.keyboard.update_hover(px, py, is_pointing=(s == G["point"]))
        if onset and s == G["pinch_index"] and self._click_allowed(t):
            key = self.keyboard.press()
            if key:
                if self.control_on: self.ctrl.press_key(key)
                self.last_click_t = t; ev.append(f"key_{key}")
        return ev


# ----------------------------------------------------------------------
# Drawing the user interface
# ----------------------------------------------------------------------
def draw_ui(frame, eng, hand, fps, logging_on, last_events):
    h, w = frame.shape[:2]
    if eng.mode == "MOUSE":
        x0, y0, x1, y1 = config.ACTIVE_REGION
        cv2.rectangle(frame, (int(x0 * w), int(y0 * h)), (int(x1 * w), int(y1 * h)), (255, 200, 0), 1)
```

```
        cv2.putText(frame, "active area", (int(x0 * w) + 4, int(y0 * h) - 6),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.45, (255, 200, 0), 1)
    else:
        eng.keyboard.draw(frame)

    if hand is not None:
        draw_hand(frame, hand)
        if eng.mode == "MOUSE":
            ax, ay = anchor_point(hand.landmarks, eng.anchor)
            cv2.circle(frame, (int(ax * w), int(ay * h)), 9, (0, 0, 255), 2)
        elif hasattr(eng, "tip_px"):
            cv2.circle(frame, eng.tip_px, 10, (0, 0, 255), 2)

    # top status bar
    cv2.rectangle(frame, (0, 0), (w, 64), (25, 25, 25), -1)
    mode_col = (0, 220, 255) if eng.mode == "MOUSE" else (255, 120, 255)
    cv2.putText(frame, eng.mode, (12, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.9, mode_col, 2)
    gname = config.GESTURES[eng.stable] if hand is not None else "no hand"
    cv2.putText(frame, f"{gname}", (150, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.85, (255, 255, 255), 2)
    # confidence bar
    cv2.rectangle(frame, (150, 40), (350, 52), (90, 90, 90), 1)
    if hand is not None:
        cv2.rectangle(frame, (150, 40), (150 + int(200 * eng.conf), 52), (0, 200, 0), -1)
    info = (f"FPS {fps:4.1f} | clf {'MLP' if eng.use_model else 'rules'} | filter {eng.filter_name}"
            f" | anchor {eng.anchor} | vote {'on' if eng.voter.enabled else 'off'}")
    cv2.putText(frame, info, (370, 22), cv2.FONT_HERSHEY_SIMPLEX, 0.45, (200, 200, 200), 1)
    ctl = "CONTROL ON" if eng.control_on else "CONTROL OFF (c)"
    cv2.putText(frame, ctl, (370, 48), cv2.FONT_HERSHEY_SIMPLEX, 0.5,
                (0, 255, 0) if eng.control_on else (0, 0, 255), 1)
    if logging_on:
        cv2.circle(frame, (w - 20, 20), 8, (0, 0, 255), -1)
        cv2.putText(frame, "LOG", (w - 62, 26), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255), 1)
    if last_events:
        cv2.putText(frame, last_events, (w - 260, 52), cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 255, 255), 2)

    # mode-switch progress (thumbs-up hold)
    if eng.thumbs_start is not None and not eng.mode_switched:
        p = min((time.perf_counter() - eng.thumbs_start) / config.MODE_SWITCH_HOLD_S, 1.0)
        cv2.ellipse(frame, (w // 2, h // 2), (40, 40), -90, 0, int(360 * p), (255, 120, 255), 6)

    # legend (mouse mode only, to keep the keyboard visible)
    if eng.mode == "MOUSE":
        y = h - 12 - 20 * len(config.GESTURES)
        cv2.rectangle(frame, (0, y - 20), (290, h), (25, 25, 25), -1)
        for i, g in enumerate(config.GESTURES):
            col = (0, 255, 255) if hand is not None and i == eng.stable else (210, 210, 210)
            cv2.putText(frame, f"{g:<13} {config.GESTURE_ACTIONS[g]}", (8, y + 20 * i),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.45, col, 1, cv2.LINE_AA)


# ---------------------------------------------------------------------
# Logging (for the experiments)
# ---------------------------------------------------------------------
LOG_COLS = ["t", "fps", "proc_ms", "hand", "raw_label", "conf", "stable_label",
            "tip_x", "tip_y", "mcp_x", "mcp_y", "palm_x", "palm_y",
            "cursor_x", "cursor_y", "mode", "filter", "anchor", "voting", "events"]


def log_row(eng, hand, t, fps, proc_ms, events):
    """One CSV row per frame: raw keypoints, raw + stable prediction, cursor, events."""
    lm = hand.landmarks if hand is not None else np.full((21, 3), np.nan)
    palm = lm[[0, 5, 17]].mean(0)
    return [f"{t:.4f}", f"{fps:.2f}", f"{proc_ms:.2f}", int(hand is not None),
            eng.raw_label, f"{eng.conf:.3f}", eng.stable,
            f"{lm[8,0]:.5f}", f"{lm[8,1]:.5f}", f"{lm[5,0]:.5f}", f"{lm[5,1]:.5f}",
            f"{palm[0]:.5f}", f"{palm[1]:.5f}",
            f"{eng.cursor[0]:.1f}", f"{eng.cursor[1]:.1f}", eng.mode,
            eng.filter_name, eng.anchor, int(eng.voter.enabled), ";".join(events)]


def open_log():
    path = os.path.join(config.LOG_DIR, time.strftime("session_%Y%m%d_%H%M%S.csv"))
    f = open(path, "w", newline="")
    w = csv.writer(f); w.writerow(LOG_COLS)
    print(f"[log] writing {path}")
    return f, w


def main():
```

```
    ap = argparse.ArgumentParser()
    ap.add_argument("--model", default=config.GESTURE_MODEL_PATH)
    ap.add_argument("--rules", action="store_true", help="use the rule-based classifier")
    ap.add_argument("--filter", default=config.DEFAULT_FILTER, choices=list(FILTERS))
    ap.add_argument("--anchor", default=config.DEFAULT_ANCHOR, choices=ANCHORS)
    ap.add_argument("--no-vote", action="store_true")
    ap.add_argument("--dry-run", action="store_true", help="do not send real mouse/keyboard events")
    ap.add_argument("--log", action="store_true")
    ap.add_argument("--camera", type=int, default=config.CAMERA_INDEX)
    args = ap.parse_args()

    predictor = None
    if not args.rules:
        if os.path.exists(args.model):
            from model import GesturePredictor
            predictor = GesturePredictor(args.model)
            print(f"Loaded gesture model {args.model}")
        else:
            print(f"No model at {args.model} -> using rule-based classifier. Run train.py first.")

    cap = cv2.VideoCapture(args.camera)
    cap.set(cv2.CAP_PROP_FRAME_WIDTH, config.FRAME_WIDTH)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, config.FRAME_HEIGHT)
    ok, frame = cap.read()
    if not ok:
        raise SystemExit("Camera not found. Try --camera 1")
    fh, fw = frame.shape[:2]

    tracker = HandTracker()
    ctrl = Controller(dry_run=args.dry_run)
    eng = GestureEngine(ctrl, predictor, args.filter, args.anchor, not args.no_vote, (fw, fh))

    win = "Gesture Mouse & Keyboard (EE604)"
    cv2.namedWindow(win, cv2.WINDOW_NORMAL)
    try:
        cv2.setWindowProperty(win, cv2.WND_PROP_TOPMOST, 1)   # keep the window above others
    except Exception:
        pass

    log_f, log_w = open_log() if args.log else (None, None)
    fps, t_prev = 0.0, time.perf_counter()
    last_ev, last_ev_t = "", 0.0

    while True:
        ok, frame = cap.read()
        if not ok:
            break
        t0 = time.perf_counter()
        frame = cv2.flip(frame, 1)                  # mirror view
        hand = tracker.process(frame)
        events = eng.step(hand, t0)
        proc_ms = (time.perf_counter() - t0) * 1000

        dt = t0 - t_prev; t_prev = t0
        fps = 0.9 * fps + 0.1 * (1.0 / dt) if dt > 0 else fps
        if events:
            last_ev, last_ev_t = events[-1], t0
        draw_ui(frame, eng, hand, fps, log_w is not None, last_ev if t0 - last_ev_t < 0.8 else "")
        cv2.imshow(win, frame)

        if log_w is not None:
            log_w.writerow(log_row(eng, hand, t0, fps, proc_ms, events))

        key = cv2.waitKey(1) & 0xFF
        if key in (ord("q"), 27):
            break
        elif key == ord("c"):
            eng.control_on = not eng.control_on
            ctrl.mouse_up()
        elif key == ord("k"):
            eng.mode = "KEYBOARD" if eng.mode == "MOUSE" else "MOUSE"
        elif key == ord("f"):
            names = list(FILTERS); eng.set_filter(names[(names.index(eng.filter_name) + 1) % len(names)])
        elif key == ord("a"):
            eng.anchor = ANCHORS[(ANCHORS.index(eng.anchor) + 1) % len(ANCHORS)]; eng.filter.reset()
        elif key == ord("v"):
            eng.voter.enabled = not eng.voter.enabled; eng.voter.reset()
        elif key == ord("r"):
            if eng.predictor is not None:
```

```
                eng.use_model = not eng.use_model
        elif key == ord("l"):
            if log_w is None:
                log_f, log_w = open_log()
            else:
                log_f.close(); log_f, log_w = None, None

    ctrl.mouse_up()
    if log_f:
        log_f.close()
    cap.release()
    tracker.close()
    cv2.destroyAllWindows()


if __name__ == "__main__":
    main()
```

collect_data.py

```
"""
collect_data.py
---------------
Record our own gesture dataset with the webcam.

Every saved sample = one frame with a detected hand:
  * a row in data/landmarks.csv : user, label, handedness, 63 keypoint values
  * a 96x96 hand crop in data/images/<label>/   (for the CNN baseline)

How to use:
    python collect_data.py --user guna
    - press 0..6 to choose the gesture (see list on screen)
    - press SPACE to start / stop recording that gesture
    - move your hand around, change distance and angle while recording
    - press q to quit
Aim for about 300-400 samples per gesture per person (~15-20 s each).
Each group member should record with their own --user name, so that we
can test on a person the model has never seen (leave-one-user-out).
"""
import argparse
import csv
import os
import time

import cv2

import config
from hand_tracker import HandTracker, draw_hand, crop_hand


def count_existing(path, user):
    """How many samples per gesture this user has already saved."""
    counts = {g: 0 for g in config.GESTURES}
    if os.path.exists(path):
        with open(path, newline="") as f:
            for row in csv.DictReader(f):
                if row["user"] == user and row["label"] in counts:
                    counts[row["label"]] += 1
    return counts


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--user", required=True, help="name of the person recording")
    ap.add_argument("--camera", type=int, default=config.CAMERA_INDEX)
    ap.add_argument("--save-images", type=int, default=1, help="1 = also save hand crops")
    ap.add_argument("--image-every", type=int, default=2, help="save a crop every N frames")
    args = ap.parse_args()

    new_file = not os.path.exists(config.LANDMARK_CSV)
    f = open(config.LANDMARK_CSV, "a", newline="")
    writer = csv.writer(f)
    if new_file:
        header = ["user", "label", "handedness"] + [f"{a}{i}" for i in range(21) for a in "xyz"]
        writer.writerow(header)

    counts = count_existing(config.LANDMARK_CSV, args.user)
    cap = cv2.VideoCapture(args.camera)
```

```
    cap.set(cv2.CAP_PROP_FRAME_WIDTH, config.FRAME_WIDTH)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, config.FRAME_HEIGHT)
    tracker = HandTracker()

    current, recording, frame_id = 0, False, 0
    print("Keys: 0-6 choose gesture | SPACE start/stop | q quit")

    while True:
        ok, frame = cap.read()
        if not ok:
            print("Camera not found."); break
        frame = cv2.flip(frame, 1)        # mirror view (feels natural, same as the app)
        clean = frame.copy()
        hand = tracker.process(frame)
        frame_id += 1

        if hand is not None:
            draw_hand(frame, hand)
            if recording:
                label = config.GESTURES[current]
                writer.writerow([args.user, label, hand.handedness] +
                                [f"{v:.5f}" for v in hand.landmarks.flatten()])
                counts[label] += 1
                if args.save_images and frame_id % args.image_every == 0:
                    crop = crop_hand(clean, hand)
                    if crop is not None:
                        d = os.path.join(config.IMAGE_DIR, label)
                        os.makedirs(d, exist_ok=True)
                        cv2.imwrite(os.path.join(d, f"{args.user}_{int(time.time()*1000)}.jpg"), crop)

        # ---------------- on-screen help ----------------
        cv2.rectangle(frame, (0, 0), (330, 30 + 24 * config.NUM_CLASSES), (30, 30, 30), -1)
        for i, g in enumerate(config.GESTURES):
            col = (0, 255, 255) if i == current else (220, 220, 220)
            cv2.putText(frame, f"{i}: {g:<13} {counts[g]:>5}", (10, 25 + 24 * i),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, col, 1, cv2.LINE_AA)
        status = "REC" if recording else "paused (SPACE)"
        cv2.putText(frame, f"user={args.user}  {status}", (10, 25 + 24 * config.NUM_CLASSES),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255) if recording else (200, 200, 200), 2)
        if hand is None:
            cv2.putText(frame, "no hand", (frame.shape[1] - 150, 30),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)
        cv2.imshow("Collect gesture data", frame)

        key = cv2.waitKey(1) & 0xFF
        if key == ord("q"):
            break
        if key == ord(" "):
            recording = not recording
            f.flush()
        if ord("0") <= key < ord("0") + config.NUM_CLASSES:
            current, recording = key - ord("0"), False

    f.close()
    cap.release()
    tracker.close()
    cv2.destroyAllWindows()
    print("Saved counts:", counts)


if __name__ == "__main__":
    main()
```

config.py

```
"""
config.py
---------
All settings of the project live in one place so that every script
(data collection, training, the main app, experiments) uses the same values.
"""
import os

# ---------------------------------------------------------------------------
# Folders
# ---------------------------------------------------------------------------
ROOT_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(ROOT_DIR, "data")
```

```
IMAGE_DIR = os.path.join(DATA_DIR, "images")          # hand crops for the CNN baseline
LANDMARK_CSV = os.path.join(DATA_DIR, "landmarks.csv")  # 21 hand keypoints per sample
MODEL_DIR = os.path.join(ROOT_DIR, "models")
RESULTS_DIR = os.path.join(ROOT_DIR, "results")
LOG_DIR = os.path.join(ROOT_DIR, "logs")

for _d in (DATA_DIR, IMAGE_DIR, MODEL_DIR, RESULTS_DIR, LOG_DIR):
    os.makedirs(_d, exist_ok=True)

# MediaPipe hand landmark model (downloaded automatically on first run)
HAND_MODEL_PATH = os.path.join(MODEL_DIR, "hand_landmarker.task")
HAND_MODEL_URL = ("https://storage.googleapis.com/mediapipe-models/hand_landmarker/"
                  "hand_landmarker/float16/latest/hand_landmarker.task")

# Trained gesture classifier
GESTURE_MODEL_PATH = os.path.join(MODEL_DIR, "gesture_mlp.pt")

# ---------------------------------------------------------------------------
# Gesture classes  (index = class id, also the key you press while collecting)
# ---------------------------------------------------------------------------
GESTURES = [
    "idle_palm",     # 0 : open palm          -> do nothing (pause / "clutch")
    "point",         # 1 : only index up      -> move cursor / hover key
    "pinch_index",   # 2 : thumb + index      -> left click / type key
    "pinch_middle",  # 3 : thumb + middle     -> right click
    "two_finger",    # 4 : index + middle up  -> scroll mode
    "fist",          # 5 : closed fist        -> drag (hold left button)
    "thumbs_up",     # 6 : thumb up (hold 1s) -> switch Mouse <-> Keyboard mode
]
NUM_CLASSES = len(GESTURES)

# What each gesture does (shown in the UI legend)
GESTURE_ACTIONS = {
    "idle_palm": "Pause",
    "point": "Move / Hover",
    "pinch_index": "Left click / Type",
    "pinch_middle": "Right click",
    "two_finger": "Scroll",
    "fist": "Drag",
    "thumbs_up": "Switch mode (hold)",
}

# ---------------------------------------------------------------------------
# Camera
# ---------------------------------------------------------------------------
CAMERA_INDEX = 0
FRAME_WIDTH = 960
FRAME_HEIGHT = 540

# Part of the camera frame that is mapped to the full screen.
# A smaller box means less arm movement is needed to reach screen corners.
ACTIVE_REGION = (0.20, 0.15, 0.80, 0.70)   # (x_min, y_min, x_max, y_max) in [0,1]

# ---------------------------------------------------------------------------
# Novelty 1: cursor smoothing (One-Euro filter by default)
# ---------------------------------------------------------------------------
DEFAULT_FILTER = "one_euro"        # one of: none, moving_avg, kalman, one_euro
ONE_EURO_MIN_CUTOFF = 1.0          # lower  -> smoother when the hand is slow
ONE_EURO_BETA = 0.02               # higher -> less lag when the hand is fast
ONE_EURO_D_CUTOFF = 1.0
MOVING_AVG_WINDOW = 5
KALMAN_PROCESS_NOISE = 5.0        # pixel^2 units
KALMAN_MEASURE_NOISE = 20.0

# ---------------------------------------------------------------------------
# Novelty 2: stable anchor point for the cursor
# "index_mcp" = knuckle of the index finger (landmark 5). It hardly moves
# when you pinch, so the cursor does not jump at the moment of a click.
# ---------------------------------------------------------------------------
DEFAULT_ANCHOR = "index_mcp"       # one of: index_tip, index_mcp, palm

# ---------------------------------------------------------------------------
# Novelty 3: confidence + temporal voting (fewer false clicks)
# ---------------------------------------------------------------------------
VOTE_WINDOW = 5          # look at the last N frames
VOTE_MIN_COUNT = 3       # a gesture must win at least this many of them
CONF_THRESHOLD = 0.70    # and have mean confidence above this

# ---------------------------------------------------------------------------
```

```
# Action timing
# ------------------------------------------------------------------------------
CLICK_COOLDOWN_S = 0.35       # minimum time between two clicks
MODE_SWITCH_HOLD_S = 1.0      # hold thumbs-up this long to switch mode
SCROLL_GAIN = 2500            # scroll amount per unit of vertical hand motion
SCROLL_DEADZONE = 0.004       # ignore tiny vertical hand motions

# ------------------------------------------------------------------------------
# Training
# ------------------------------------------------------------------------------
SEED = 42
EPOCHS = 60
BATCH_SIZE = 64
LEARNING_RATE = 1e-3
AUGMENT_COPIES = 4            # extra augmented copies of every training sample
```

controller.py

```
"""
controller.py
-------------
Sends the real mouse / keyboard events to the operating system (pyautogui).
With dry_run=True nothing is sent (useful for testing and for logging-only runs).
"""


class Controller:
    def __init__(self, dry_run=False):
        self.dry_run = dry_run
        self.mouse_is_down = False
        if dry_run:
            self.screen_w, self.screen_h = 1920, 1080
            return
        import pyautogui
        pyautogui.FAILSAFE = False   # do not abort when the cursor reaches a screen corner
        pyautogui.PAUSE = 0          # no extra delay after each call (we need real time)
        self.pg = pyautogui
        self.screen_w, self.screen_h = pyautogui.size()

    # ---------------- mouse ----------------
    def move(self, x, y):
        x = min(max(int(x), 0), self.screen_w - 1)
        y = min(max(int(y), 0), self.screen_h - 1)
        if not self.dry_run:
            self.pg.moveTo(x, y)

    def click(self, button="left"):
        if not self.dry_run:
            self.pg.click(button=button)

    def mouse_down(self):
        if not self.mouse_is_down and not self.dry_run:
            self.pg.mouseDown()
        self.mouse_is_down = True

    def mouse_up(self):
        if self.mouse_is_down and not self.dry_run:
            self.pg.mouseUp()
        self.mouse_is_down = False

    def scroll(self, amount):
        if amount and not self.dry_run:
            self.pg.scroll(int(amount))

    # ---------------- keyboard ----------------
    def press_key(self, key):
        """key: a single character, or 'space' / 'backspace' / 'enter'."""
        if not self.dry_run:
            self.pg.press(key.lower() if len(key) == 1 else key)
```

features.py

```
"""
features.py
-----------
Turns the 21 raw hand keypoints into a feature vector for the classifier.
```

```
"""

Why not just use raw (x, y) values?  Raw values change when the hand moves
to another part of the image, comes closer to the camera, or tilts.
The gesture is the same, so the features should be the same. We therefore:

  1. Mirror left hands so they look like right hands   (left/right invariance)
  2. Move the wrist to the origin                       (position invariance)
  3. Divide by palm size (wrist -> middle knuckle)      (scale / distance invariance)
  4. Rotate so the palm always points "up"              (in-plane rotation invariance)
  5. Add hand-shape features: joint bend angles and fingertip distances

Feature modes (used in the ablation study in train.py):
  "raw"        : 63 raw values
  "norm"       : steps 1-3        (63 values)
  "norm_rot"   : steps 1-4        (63 values)
  "full"       : steps 1-5        (63 + 15 angles + 10 distances = 88 values)
"""
import numpy as np

FEATURE_MODES = ["raw", "norm", "norm_rot", "full"]

FINGERTIPS = [4, 8, 12, 16, 20]
# (a, b, c) triples: angle at joint b between bones b->a and b->c
JOINT_TRIPLES = [
    (1, 2, 3), (2, 3, 4), (0, 1, 2),             # thumb
    (0, 5, 6), (5, 6, 7), (6, 7, 8),             # index
    (0, 9, 10), (9, 10, 11), (10, 11, 12),       # middle
    (0, 13, 14), (13, 14, 15), (14, 15, 16),     # ring
    (0, 17, 18), (17, 18, 19), (18, 19, 20),     # pinky
]


def normalize_landmarks(lm, handedness="Right", rotate=True):
    """Steps 1-4 above. lm: (21, 3) array. Returns a new (21, 3) array."""
    p = np.asarray(lm, dtype=np.float32).copy()

    # 1. mirror left hand
    if handedness == "Left":
        p[:, 0] = 1.0 - p[:, 0]

    # 2. wrist at origin
    p -= p[0]

    # 3. scale by palm size
    palm = np.linalg.norm(p[9, :2]) + 1e-6
    p /= palm

    # 4. rotate (in x-y plane) so wrist -> middle knuckle points straight up (-y)
    if rotate:
        # (after scaling, |p[9]| in x-y is 1). Rotation matrix [[c,-s],[s,c]]
        # with c=-vy, s=-vx maps (vx, vy) to (0, -1), i.e. "up" in image coords.
        vx, vy = p[9, 0], p[9, 1]
        r = np.hypot(vx, vy) + 1e-6
        c, s = -vy / r, -vx / r
        x, y = p[:, 0].copy(), p[:, 1].copy()
        p[:, 0] = c * x - s * y
        p[:, 1] = s * x + c * y
    return p


def joint_angles(p):
    """15 bend angles (radians) at the finger joints."""
    out = []
    for a, b, c in JOINT_TRIPLES:
        v1, v2 = p[a] - p[b], p[c] - p[b]
        cosang = np.dot(v1, v2) / (np.linalg.norm(v1) * np.linalg.norm(v2) + 1e-6)
        out.append(np.arccos(np.clip(cosang, -1.0, 1.0)))
    return np.array(out, dtype=np.float32)


def fingertip_distances(p):
    """10 distances between every pair of fingertips (already scale-normalized)."""
    out = []
    for i in range(len(FINGERTIPS)):
        for j in range(i + 1, len(FINGERTIPS)):
            out.append(np.linalg.norm(p[FINGERTIPS[i]] - p[FINGERTIPS[j]]))
    return np.array(out, dtype=np.float32)
```

```python
def extract_features(lm, handedness="Right", mode="full"):
    """Main entry point. Returns a 1-D float32 feature vector."""
    lm = np.asarray(lm, dtype=np.float32).reshape(21, 3)
    if mode == "raw":
        return lm.flatten()
    if mode == "norm":
        return normalize_landmarks(lm, handedness, rotate=False).flatten()
    p = normalize_landmarks(lm, handedness, rotate=True)
    if mode == "norm_rot":
        return p.flatten()
    if mode == "full":
        return np.concatenate([p.flatten(), joint_angles(p), fingertip_distances(p)])
    raise ValueError(f"Unknown feature mode: {mode}")


def feature_dim(mode):
    return {"raw": 63, "norm": 63, "norm_rot": 63, "full": 88}[mode]


def augment_landmarks(lm, rng):
    """
    Data augmentation on raw keypoints (training only):
    small random rotation, scaling, shift and noise. This makes the model
    robust to how different people hold their hand.
    """
    p = np.asarray(lm, dtype=np.float32).reshape(21, 3).copy()
    center = p[0, :2].copy()
    ang = np.deg2rad(rng.uniform(-20, 20))
    c, s = np.cos(ang), np.sin(ang)
    xy = p[:, :2] - center
    xy = xy @ np.array([[c, -s], [s, c]], dtype=np.float32).T
    xy *= rng.uniform(0.85, 1.15)
    p[:, :2] = xy + center + rng.uniform(-0.05, 0.05, size=2)
    p += rng.normal(0, 0.004, size=p.shape).astype(np.float32)
    return p
```

filters.py

```python
"""
filters.py
----------
Cursor smoothing filters. Hand keypoints shake a little in every frame
(jitter), so a raw cursor is hard to use for small targets. We compare:

  none        : raw position
  moving_avg  : mean of the last N positions (smooth but laggy)
  kalman      : constant-velocity Kalman filter
  one_euro    : One-Euro filter (Casiez et al., CHI 2012) - low cutoff when the
                hand is slow (removes jitter), high cutoff when fast (removes lag).
                This speed-adaptive behaviour is what we use by default.

All filters work on 2-D points in SCREEN PIXELS and take the time stamp t (seconds).
(The One-Euro beta and the Kalman noise values in config.py assume pixel units.)
"""
import math
from collections import deque

import numpy as np

import config


class NoFilter:
    name = "none"

    def __call__(self, x, y, t):
        return x, y

    def reset(self):
        pass


class MovingAverage:
    name = "moving_avg"

    def __init__(self, window=config.MOVING_AVG_WINDOW):
        self.buf = deque(maxlen=window)

```

```
    def __call__(self, x, y, t):
        self.buf.append((x, y))
        a = np.mean(self.buf, axis=0)
        return float(a[0]), float(a[1])

    def reset(self):
        self.buf.clear()


class Kalman2D:
    """State = [x, y, vx, vy], constant-velocity model."""
    name = "kalman"

    def __init__(self, q=config.KALMAN_PROCESS_NOISE, r=config.KALMAN_MEASURE_NOISE):
        self.q, self.r = q, r
        self.reset()

    def reset(self):
        self.s = None
        self.P = np.eye(4)
        self.t = None

    def __call__(self, x, y, t):
        z = np.array([x, y])
        if self.s is None:
            self.s = np.array([x, y, 0.0, 0.0])
            self.t = t
            return x, y
        dt = max(t - self.t, 1e-3)
        self.t = t
        F = np.array([[1, 0, dt, 0], [0, 1, 0, dt], [0, 0, 1, 0], [0, 0, 0, 1]])
        H = np.array([[1, 0, 0, 0], [0, 1, 0, 0]])
        Q = self.q * np.eye(4)
        R = self.r * np.eye(2)
        # predict
        self.s = F @ self.s
        self.P = F @ self.P @ F.T + Q
        # update
        y_res = z - H @ self.s
        S = H @ self.P @ H.T + R
        K = self.P @ H.T @ np.linalg.inv(S)
        self.s = self.s + K @ y_res
        self.P = (np.eye(4) - K @ H) @ self.P
        return float(self.s[0]), float(self.s[1])


class _LowPass:
    def __init__(self):
        self.y = None

    def __call__(self, x, alpha):
        self.y = x if self.y is None else alpha * x + (1 - alpha) * self.y
        return self.y


class OneEuro1D:
    def __init__(self, min_cutoff, beta, d_cutoff):
        self.min_cutoff, self.beta, self.d_cutoff = min_cutoff, beta, d_cutoff
        self.x_f, self.dx_f = _LowPass(), _LowPass()
        self.t_prev, self.x_prev = None, None

    @staticmethod
    def _alpha(cutoff, dt):
        tau = 1.0 / (2 * math.pi * cutoff)
        return 1.0 / (1.0 + tau / dt)

    def __call__(self, x, t):
        if self.t_prev is None:
            self.t_prev, self.x_prev = t, x
            self.dx_f(0.0, 1.0)
            return self.x_f(x, 1.0)
        dt = max(t - self.t_prev, 1e-3)
        dx = (x - self.x_prev) / dt                         # speed
        edx = self.dx_f(dx, self._alpha(self.d_cutoff, dt))
        cutoff = self.min_cutoff + self.beta * abs(edx)    # faster hand -> higher cutoff
        out = self.x_f(x, self._alpha(cutoff, dt))
        self.t_prev, self.x_prev = t, x
        return out

```

```

class OneEuro:
    name = "one_euro"

    def __init__(self, min_cutoff=config.ONE_EURO_MIN_CUTOFF, beta=config.ONE_EURO_BETA,
                 d_cutoff=config.ONE_EURO_D_CUTOFF):
        self.args = (min_cutoff, beta, d_cutoff)
        self.reset()

    def reset(self):
        self.fx, self.fy = OneEuro1D(*self.args), OneEuro1D(*self.args)

    def __call__(self, x, y, t):
        return self.fx(x, t), self.fy(y, t)


FILTERS = {"none": NoFilter, "moving_avg": MovingAverage, "kalman": Kalman2D, "one_euro": OneEuro}


def make_filter(name):
    return FILTERS[name]()
```

hand_tracker.py

```
"""
hand_tracker.py
---------------
Thin wrapper around the MediaPipe "Hand Landmarker" (Tasks API).

For each camera frame it returns the 21 hand keypoints (x, y, z), whether
it is a left or right hand, and a bounding box. We build on this existing
model (allowed by the course rules); our own work starts after this step.

Landmark numbering (MediaPipe):
    0 wrist
    1-4   thumb  (CMC, MCP, IP, TIP)
    5-8   index  (MCP, PIP, DIP, TIP)
    9-12  middle
    13-16 ring
    17-20 pinky
"""
import os
import time
import urllib.request

import cv2
import numpy as np
import mediapipe as mp
from mediapipe.tasks import python as mp_python
from mediapipe.tasks.python import vision

import config

# Pairs of landmarks joined by a line when drawing the hand skeleton
HAND_CONNECTIONS = [
    (0, 1), (1, 2), (2, 3), (3, 4),          # thumb
    (0, 5), (5, 6), (6, 7), (7, 8),          # index
    (5, 9), (9, 10), (10, 11), (11, 12),     # middle
    (9, 13), (13, 14), (14, 15), (15, 16),   # ring
    (13, 17), (17, 18), (18, 19), (19, 20),  # pinky
    (0, 17),                                 # palm base
]


def ensure_hand_model(path=config.HAND_MODEL_PATH, url=config.HAND_MODEL_URL):
    """Download the MediaPipe hand model the first time (about 7.5 MB)."""
    if not os.path.exists(path):
        print(f"[hand_tracker] Downloading hand model to {path} ...")
        tmp = path + ".part"                     # avoid a broken file if the download stops
        urllib.request.urlretrieve(url, tmp)
        os.replace(tmp, path)
        print("[hand_tracker] Done.")
    return path


class HandResult:
    """Simple container for one detected hand."""

```

```
    def __init__(self, landmarks, handedness, score, frame_shape):
        self.landmarks = landmarks          # (21, 3) array, x,y in [0,1], z relative depth
        self.handedness = handedness        # "Left" or "Right"
        self.score = score                  # detection confidence
        h, w = frame_shape[:2]
        xs, ys = landmarks[:, 0] * w, landmarks[:, 1] * h
        self.bbox = (int(xs.min()), int(ys.min()), int(xs.max()), int(ys.max()))

    def pixel(self, idx, frame_shape):
        """Pixel coordinates of landmark `idx`."""
        h, w = frame_shape[:2]
        return int(self.landmarks[idx, 0] * w), int(self.landmarks[idx, 1] * h)


class HandTracker:
    """Detects at most one hand per frame using MediaPipe in VIDEO mode."""

    def __init__(self, num_hands=1, det_conf=0.5, track_conf=0.5):
        model_path = ensure_hand_model()
        options = vision.HandLandmarkerOptions(
            base_options=mp_python.BaseOptions(model_asset_path=model_path),
            running_mode=vision.RunningMode.VIDEO,   # uses tracking between frames -> faster
            num_hands=num_hands,
            min_hand_detection_confidence=det_conf,
            min_hand_presence_confidence=det_conf,
            min_tracking_confidence=track_conf,
        )
        self.landmarker = vision.HandLandmarker.create_from_options(options)
        self._last_ts = -1

    def process(self, frame_bgr):
        """Run detection on one BGR frame. Returns HandResult or None."""
        rgb = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB)
        mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb)

        # VIDEO mode needs strictly increasing timestamps (in ms)
        ts = int(time.monotonic() * 1000)
        if ts <= self._last_ts:
            ts = self._last_ts + 1
        self._last_ts = ts

        result = self.landmarker.detect_for_video(mp_image, ts)
        if not result.hand_landmarks:
            return None

        lms = result.hand_landmarks[0]
        landmarks = np.array([[p.x, p.y, p.z] for p in lms], dtype=np.float32)
        category = result.handedness[0][0]
        return HandResult(landmarks, category.category_name, category.score, frame_bgr.shape)

    def close(self):
        self.landmarker.close()


def draw_hand(frame, hand, color=(0, 255, 0)):
    """Draw the hand skeleton and keypoints on the frame (in place)."""
    pts = [hand.pixel(i, frame.shape) for i in range(21)]
    for a, b in HAND_CONNECTIONS:
        cv2.line(frame, pts[a], pts[b], color, 2, cv2.LINE_AA)
    for i, p in enumerate(pts):
        r = 6 if i in (4, 8, 12, 16, 20) else 4      # fingertips a little bigger
        cv2.circle(frame, p, r, (255, 255, 255), -1, cv2.LINE_AA)
        cv2.circle(frame, p, r, color, 1, cv2.LINE_AA)


def crop_hand(frame, hand, size=96, margin=0.25):
    """Square crop around the hand (used to build the image dataset for the CNN baseline)."""
    h, w = frame.shape[:2]
    x1, y1, x2, y2 = hand.bbox
    side = int(max(x2 - x1, y2 - y1) * (1 + 2 * margin))
    cx, cy = (x1 + x2) // 2, (y1 + y2) // 2
    x1, y1 = max(cx - side // 2, 0), max(cy - side // 2, 0)
    x2, y2 = min(x1 + side, w), min(y1 + side, h)
    crop = frame[y1:y2, x1:x2]
    if crop.size == 0:
        return None
    return cv2.resize(crop, (size, size), interpolation=cv2.INTER_AREA)
```

make_figures.py

```python
"""
make_figures.py
---------------
Draws the figures for the report and the slides (saved in results/figures/):

  overview.pdf/.png   pipeline of the proposed method (no data needed)
  gestures.png        one real hand crop + its normalized keypoints per gesture
  features.png        raw keypoints -> normalized -> rotated (one real sample)
  anchor.png          why the knuckle anchor is stable during a pinch

The last three use the recorded dataset (data/landmarks.csv, data/images/).
"""
import glob
import os

import cv2
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch

import config
from features import normalize_landmarks
from hand_tracker import HAND_CONNECTIONS

FIG_DIR = os.path.join(config.RESULTS_DIR, "figures")
os.makedirs(FIG_DIR, exist_ok=True)

OURS = "#d9ecff"       # colour of our blocks
OURS_EDGE = "#1f5fa8"
EXIST = "#eeeeee"      # colour of existing blocks
EXIST_EDGE = "#777777"


# ---------------------------------------------------------------------
# 1. Overview figure
# ---------------------------------------------------------------------
def box(ax, x, y, w, h, title, body, ours=True, fs=7.2):
    ax.add_patch(FancyBboxPatch((x, y), w, h, boxstyle="round,pad=0.02,rounding_size=1.2",
                                fc=OURS if ours else EXIST, ec=OURS_EDGE if ours else EXIST_EDGE, lw=1.2))
    ax.text(x + w / 2, y + h - 2.2, title, ha="center", va="top", fontsize=fs + 0.8, weight="bold")
    ax.text(x + w / 2, y + h - 6.3, body, ha="center", va="top", fontsize=fs, linespacing=1.25)


def arrow(ax, x1, y1, x2, y2, dashed=False):
    ax.annotate("", xy=(x2, y2), xytext=(x1, y1),
                arrowprops=dict(arrowstyle="-|>", lw=1.3, color="#333333",
                                linestyle="--" if dashed else "-", shrinkA=0, shrinkB=0))


def overview():
    fig, ax = plt.subplots(figsize=(7.2, 2.55))
    ax.set_xlim(0, 144); ax.set_ylim(0, 51); ax.axis("off")
    H, Y = 21, 22
    # webcam icon-like box
    box(ax, 0, Y, 15, H, "Webcam", "RGB frame\n960x540\n~30 fps", ours=False)
    box(ax, 18, Y, 19, H, "MediaPipe", "hand landmarker\n21 keypoints\n(x, y, z)", ours=False)
    box(ax, 40, Y, 21, H, "Invariant\nfeatures", "\nmirror, translate,\nscale, rotate\n+15 angles +10 dists", fs=6.6)
    box(ax, 64, Y, 18, H, "Gesture\nMLP", "\n88-128-64-7\n7 gestures\n+ confidence", fs=6.6)
    box(ax, 85, Y, 17, H, "Temporal\nvoting", "\nN=5 frames,\nk=3 votes,\nconf >= 0.7", fs=6.6)
    # action branches
    box(ax, 105, 33, 26, 17, "Mouse mode", "knuckle anchor ->\nOne-Euro filter -> cursor\nclick, scroll, drag", fs=6.4)
    box(ax, 105, 13, 26, 17, "Keyboard mode", "air keyboard\nwith hover-lock\npinch = type key", fs=6.4)
    box(ax, 133.5, Y, 10.5, H, "OS", "\nmouse &\nkeyboard\nevents", ours=False, fs=6.4)
    for x1, x2 in ((15, 18), (37, 40), (61, 64), (82, 85)):
        arrow(ax, x1, Y + H / 2, x2, Y + H / 2)
    arrow(ax, 102, Y + H / 2, 105, 41.5); arrow(ax, 102, Y + H / 2, 105, 21.5)
    arrow(ax, 131, 41.5, 133.5, Y + H / 2 + 3); arrow(ax, 131, 21.5, 133.5, Y + H / 2 - 3)
    ax.text(118, 31.5, "switch mode: thumbs-up, hold 1 s", fontsize=5.4, ha="center", va="center", color="#555555")
    # personalization
    box(ax, 56, 1.5, 34, 15, "Few-shot personalization", "retrain only the last layer\nwith K samples / gesture", fs=6.4)
    arrow(ax, 73, 16.5, 73, Y, dashed=True)
    # legend
    ax.add_patch(FancyBboxPatch((1, 4), 5, 4, boxstyle="round,pad=0.1", fc=EXIST, ec=EXIST_EDGE))
```

```
    ax.text(7.5, 6, "existing component", fontsize=6.5, va="center")
    ax.add_patch(FancyBboxPatch((1, 11), 5, 4, boxstyle="round,pad=0.1", fc=OURS, ec=OURS_EDGE))
    ax.text(7.5, 13, "proposed (ours)", fontsize=6.5, va="center")
    fig.tight_layout(pad=0.1)
    for ext in ("pdf", "png"):
        fig.savefig(os.path.join(FIG_DIR, f"overview.{ext}"), dpi=300)
    plt.close(fig)
    print("saved overview")


# ---------------------------------------------------------------------
# Helpers that need data
# ---------------------------------------------------------------------
def load():
    from train import load_dataset
    if not os.path.exists(config.LANDMARK_CSV):
        return None
    return load_dataset()


def medoid_index(lms, hands, idx):
    """Sample closest to the class mean (a 'typical' example)."""
    F = np.stack([normalize_landmarks(lms[i], hands[i]).flatten() for i in idx])
    return idx[np.argmin(np.linalg.norm(F - F.mean(0), axis=1))]


def draw_skel(ax, p, color="#1f5fa8", lw=1.6, ms=10):
    for a, b in HAND_CONNECTIONS:
        ax.plot([p[a, 0], p[b, 0]], [p[a, 1], p[b, 1]], color=color, lw=lw)
    ax.scatter(p[:, 0], p[:, 1], s=ms, color="white", edgecolor=color, zorder=3, lw=0.8)


def gestures(data):
    users, labels, hands, lms = data
    n = config.NUM_CLASSES
    fig, axes = plt.subplots(2, n, figsize=(7.2, 2.45))
    for c, g in enumerate(config.GESTURES):
        idx = np.where(labels == c)[0]
        ax_img, ax_sk = axes[0, c], axes[1, c]
        imgs = sorted(glob.glob(os.path.join(config.IMAGE_DIR, g, "*.jpg")))
        if imgs:
            im = cv2.imread(imgs[len(imgs) // 2])
            ax_img.imshow(im[:, :, ::-1])
        ax_img.set_title(g.replace("_", " "), fontsize=7)
        ax_img.axis("off")
        if len(idx):
            p = normalize_landmarks(lms[medoid_index(lms, hands, idx)], hands[idx[0]])
            draw_skel(ax_sk, p)
        ax_sk.set_xlim(-1.6, 1.6); ax_sk.set_ylim(1.0, -2.6); ax_sk.set_aspect("equal"); ax_sk.axis("off")
    fig.tight_layout(pad=0.2)
    fig.savefig(os.path.join(FIG_DIR, "gestures.png"), dpi=300); plt.close(fig)
    print("saved gestures")


def features_fig(data):
    users, labels, hands, lms = data
    idx = np.where(labels == config.GESTURES.index("point"))[0]
    if not len(idx):
        return
    i = idx[len(idx) // 3]
    raw, h = lms[i], hands[i]
    steps = [("(a) raw keypoints (image coords)", raw[:, :2], (0, 1), True),
             ("(b) mirror + translate + scale", normalize_landmarks(raw, h, rotate=False)[:, :2], None, True),
             ("(c) + rotate palm to 'up'", normalize_landmarks(raw, h, rotate=True)[:, :2], None, True)]
    fig, axes = plt.subplots(1, 3, figsize=(7.2, 2.4))
    for ax, (t, p, lim, inv) in zip(axes, steps):
        draw_skel(ax, p)
        ax.set_title(t, fontsize=7.5)
        if lim:
            ax.set_xlim(*lim); ax.set_ylim(1, 0)
        else:
            ax.set_xlim(-1.8, 1.8); ax.set_ylim(1.0, -2.6)
            ax.axhline(0, color="0.8", lw=0.6); ax.axvline(0, color="0.8", lw=0.6)
        ax.set_aspect("equal"); ax.tick_params(labelsize=6)
    fig.tight_layout(pad=0.3)
    fig.savefig(os.path.join(FIG_DIR, "features.png"), dpi=300); plt.close(fig)
    print("saved features")
```

```
def anchor_fig(data):
    users, labels, hands, lms = data
    P, Q = config.GESTURES.index("point"), config.GESTURES.index("pinch_index")
    u = users[0]
    ip, iq = np.where((labels == P) & (users == u))[0], np.where((labels == Q) & (users == u))[0]
    if not len(ip) or not len(iq):
        return
    a = normalize_landmarks(lms[medoid_index(lms, hands, ip)], hands[ip[0]])
    b = normalize_landmarks(lms[medoid_index(lms, hands, iq)], hands[iq[0]])
    fig, ax = plt.subplots(figsize=(3.3, 2.6))
    draw_skel(ax, a, color="#999999", lw=1.2); draw_skel(ax, b, color="#1f5fa8", lw=1.4)
    for k, name, col in ((8, "fingertip", "#d62728"), (5, "knuckle (ours)", "#2ca02c")):
        ax.annotate("", xy=b[k, :2], xytext=a[k, :2],
                    arrowprops=dict(arrowstyle="-|>", color=col, lw=1.8))
        d = np.linalg.norm(b[k, :2] - a[k, :2])
        ax.text(a[k, 0] + 0.15, a[k, 1], f"{name}\nmoves {d:.2f}", color=col, fontsize=6.5)
    ax.plot([], [], color="#999999", label="point"); ax.plot([], [], color="#1f5fa8", label="pinch")
    ax.legend(fontsize=6.5, loc="lower left")
    ax.set_xlim(-1.8, 2.0); ax.set_ylim(0.6, -2.6); ax.set_aspect("equal"); ax.axis("off")
    ax.set_title("Motion of anchor points from point to pinch\n(units: palm size)", fontsize=7.5)
    fig.tight_layout(pad=0.2)
    fig.savefig(os.path.join(FIG_DIR, "anchor.png"), dpi=300); plt.close(fig)
    print("saved anchor")


def main():
    overview()
    data = load()
    if data is None:
        print("No dataset yet: only the overview figure was made.")
        return
    gestures(data)
    features_fig(data)
    anchor_fig(data)


if __name__ == "__main__":
    main()
```

model.py

```
"""
model.py
--------
The deep-learning part: a small multi-layer perceptron (MLP) that classifies
the hand-keypoint feature vector into one of the gesture classes.

Why an MLP on keypoints instead of a CNN on images?
  * the input is only 88 numbers, so the network is tiny (~20k parameters)
  * it runs in well under 1 ms on a CPU, leaving time for real-time control
  * it ignores background, skin colour and lighting (MediaPipe already handled that)
The CNN-on-images baseline in train_cnn.py lets us check this claim.
"""
import json

import numpy as np
import torch
import torch.nn as nn

import config
from features import extract_features


class GestureMLP(nn.Module):
    def __init__(self, in_dim, num_classes, hidden=(128, 64), dropout=0.2):
        super().__init__()
        layers, prev = [], in_dim
        for h in hidden:
            layers += [nn.Linear(prev, h), nn.BatchNorm1d(h), nn.ReLU(), nn.Dropout(dropout)]
            prev = h
        self.body = nn.Sequential(*layers)
        self.head = nn.Linear(prev, num_classes)    # last layer (re-trained in personalization)

    def forward(self, x):
        return self.head(self.body(x))


def count_params(model):
```

```
    return sum(p.numel() for p in model.parameters())


def save_model(model, path, feature_mode, mean, std, extra=None):
    """Save weights + everything needed to use the model later."""
    torch.save({
        "state_dict": model.state_dict(),
        "in_dim": model.body[0].in_features,
        "hidden": [m.out_features for m in model.body if isinstance(m, nn.Linear)],
        "num_classes": model.head.out_features,
        "feature_mode": feature_mode,
        "mean": np.asarray(mean, dtype=np.float32).tolist(),
        "std": np.asarray(std, dtype=np.float32).tolist(),
        "classes": config.GESTURES,
        "extra": extra or {},
    }, path)


def load_checkpoint(path):
    ckpt = torch.load(path, map_location="cpu", weights_only=False)
    model = GestureMLP(ckpt["in_dim"], ckpt["num_classes"], hidden=tuple(ckpt["hidden"]))
    model.load_state_dict(ckpt["state_dict"])
    model.eval()
    return model, ckpt


class GesturePredictor:
    """Used by the live app: landmarks in -> (class index, confidence) out."""

    def __init__(self, path=config.GESTURE_MODEL_PATH):
        self.model, ckpt = load_checkpoint(path)
        self.mode = ckpt["feature_mode"]
        self.mean = np.array(ckpt["mean"], dtype=np.float32)
        self.std = np.array(ckpt["std"], dtype=np.float32)
        torch.set_num_threads(1)     # tiny model: 1 thread is fastest

    @torch.no_grad()
    def predict(self, landmarks, handedness="Right"):
        f = extract_features(landmarks, handedness, self.mode)
        x = torch.from_numpy(((f - self.mean) / self.std)[None, :].astype(np.float32))
        prob = torch.softmax(self.model(x), dim=1)[0].numpy()
        k = int(prob.argmax())
        return k, float(prob[k])
```

personalize.py

```
"""
personalize.py
--------------
Novelty 4: quick per-user calibration (few-shot personalization).

Every person makes gestures a little differently. Instead of re-training the
whole network, we keep the learned feature layers fixed and re-train ONLY the
last layer using K samples per gesture from the new user. This takes about
one second on a CPU.

Experiment mode (for the report):
    python personalize.py --user guna
  - base model is trained WITHOUT guna's data
  - guna's data is split in half: first half = calibration pool, second half = test
  - we report test accuracy for K = 0 (no calibration), 5, 10, 20, 50 samples/gesture
  - outputs results/personalization_<user>.md and .png

Deploy mode (for the demo):
    python personalize.py --user guna --deploy
  - fine-tunes the current models/gesture_mlp.pt on ALL of guna's samples
  - saves models/gesture_mlp_guna.pt  (run the app with --model models/gesture_mlp_guna.pt)
"""
import argparse
import copy
import os

import numpy as np
import torch
import torch.nn as nn
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
```

```

import config
from model import load_checkpoint, save_model
from train import load_dataset, featurize, run_mlp_experiment, augment

K_LIST = [0, 5, 10, 20, 50]


def finetune_head(model, X, y, epochs=100, lr=5e-3):
    """Re-train only the last linear layer; the body (feature extractor) is frozen."""
    model = copy.deepcopy(model)
    for p in model.body.parameters():
        p.requires_grad = False
    model.eval()                            # keep BatchNorm/Dropout in inference mode
    opt = torch.optim.Adam(model.head.parameters(), lr=lr)
    loss_fn = nn.CrossEntropyLoss()
    Xt, yt = torch.from_numpy(X), torch.from_numpy(y).long()
    with torch.no_grad():
        H = model.body(Xt)                  # features computed once (fast)
    for _ in range(epochs):
        opt.zero_grad()
        loss_fn(model.head(H), yt).backward()
        opt.step()
    return model


def pick_k_per_class(y, k, rng):
    idx = []
    for c in np.unique(y):
        cand = np.where(y == c)[0]
        idx += rng.choice(cand, size=min(k, len(cand)), replace=False).tolist()
    return np.array(idx)


def experiment(user, repeats=5):
    users, labels, hands, lms = load_dataset()
    tr, te_all = np.where(users != user)[0], np.where(users == user)[0]
    if len(te_all) == 0 or len(tr) == 0:
        raise SystemExit("Need data from this user AND at least one other user.")

    print(f"Training base model without '{user}' ({len(tr)} samples) ...")
    # base model: trained on other users (te arguments only used for its own metrics)
    model, mean, std, m, _, _ = run_mlp_experiment(lms[tr], hands[tr], labels[tr],
                                                   lms[te_all], hands[te_all], labels[te_all],
                                                   "full", config.AUGMENT_COPIES)
    norm = lambda X: ((X - mean) / std).astype(np.float32)

    # split this user's data in time order: first half = calibration pool, second half = test
    pool, test = [], []
    for c in range(config.NUM_CLASSES):
        ids = te_all[labels[te_all] == c]
        pool += ids[: len(ids) // 2].tolist()
        test += ids[len(ids) // 2:].tolist()
    pool, test = np.array(pool), np.array(test)
    Xte = torch.from_numpy(norm(featurize(lms[test], hands[test], "full")))
    yte = labels[test]

    rows = []
    for k in K_LIST:
        accs = []
        for r in range(repeats if k > 0 else 1):
            if k == 0:
                mdl = model
            else:
                rng = np.random.default_rng(r)
                sel = pool[pick_k_per_class(labels[pool], k, rng)]
                a_lm, a_h, a_y = augment(lms[sel], hands[sel], labels[sel], 4, seed=r)
                mdl = finetune_head(model, norm(featurize(a_lm, a_h, "full")), a_y)
            with torch.no_grad():
                accs.append(float((mdl(Xte).argmax(1).numpy() == yte).mean()))
        rows.append((k, np.mean(accs), np.std(accs)))
        print(f"  K={k:3d} samples/gesture -> accuracy {np.mean(accs)*100:.2f} +/- {np.std(accs)*100:.2f}%")

    md = os.path.join(config.RESULTS_DIR, f"personalization_{user}.md")
    with open(md, "w") as f:
        f.write(f"| Calibration samples per gesture (user {user}) | Accuracy (%) |\n|---|---|\n")
        for k, a, s in rows:
            f.write(f"| {k} | {a*100:.2f} +/- {s*100:.2f} |\n")
    fig, ax = plt.subplots(figsize=(4.5, 3.2))
```

```
    ax.errorbar([r[0] for r in rows], [r[1] * 100 for r in rows], yerr=[r[2] * 100 for r in rows],
                marker="o", capsize=3)
    ax.set_xlabel("calibration samples per gesture (K)"); ax.set_ylabel("accuracy on new user (%)")
    ax.set_title(f"Few-shot personalization ({user})"); ax.grid(alpha=0.3)
    fig.tight_layout(); fig.savefig(os.path.join(config.RESULTS_DIR, f"personalization_{user}.png"), dpi=200)
    print(f"Saved {md}")


def deploy(user):
    users, labels, hands, lms = load_dataset()
    sel = np.where(users == user)[0]
    if len(sel) == 0:
        raise SystemExit(f"No samples for {user}. Record some with collect_data.py --user {user}")
    model, ckpt = load_checkpoint(config.GESTURE_MODEL_PATH)
    mean, std = np.array(ckpt["mean"]), np.array(ckpt["std"])
    a_lm, a_h, a_y = augment(lms[sel], hands[sel], labels[sel], 4)
    X = ((featurize(a_lm, a_h, "full") - mean) / std).astype(np.float32)
    model = finetune_head(model, X, a_y)
    out = os.path.join(config.MODEL_DIR, f"gesture_mlp_{user}.pt")
    save_model(model, out, "full", mean, std, extra={"personalized_for": user})
    print(f"Saved personalized model to {out}")


if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--user", required=True)
    ap.add_argument("--deploy", action="store_true")
    a = ap.parse_args()
    deploy(a.user) if a.deploy else experiment(a.user)
```

rules.py

```
"""
rules.py
--------
Hand-written rule-based gesture classifier ("which fingers are up?").
This is the classic approach used by most open-source gesture-mouse projects.
We use it as a BASELINE to compare against our learned classifier,
and as a fallback so the app still runs before any model is trained.
"""
import numpy as np

import config
from features import normalize_landmarks

PINCH_THRESHOLD = 0.35   # fingertip distance / palm size


def fingers_up(p):
    """p: normalized landmarks (palm pointing up, -y is up). Returns [thumb, index, middle, ring, pinky]."""
    up = []
    # thumb: tip far from the index knuckle (sideways)
    up.append(np.linalg.norm(p[4, :2] - p[5, :2]) > 0.55)
    # other fingers: tip farther from wrist than the PIP joint
    for tip, pip in ((8, 6), (12, 10), (16, 14), (20, 18)):
        up.append(np.linalg.norm(p[tip, :2]) > np.linalg.norm(p[pip, :2]) * 1.05)
    return up


def classify_rules(landmarks, handedness="Right"):
    """Returns (label_index, confidence). Confidence is always 1.0 for rules."""
    p = normalize_landmarks(landmarks, handedness, rotate=True)
    thumb, index, middle, ring, pinky = fingers_up(p)
    d_ti = np.linalg.norm(p[4] - p[8])
    d_tm = np.linalg.norm(p[4] - p[12])
    G = config.GESTURES.index

    if d_ti < PINCH_THRESHOLD and middle:
        return G("pinch_index"), 1.0
    if d_tm < PINCH_THRESHOLD and (ring or pinky):   # ring/pinky up, so not a resting thumb
        return G("pinch_middle"), 1.0
    if index and middle and not ring and not pinky:
        return G("two_finger"), 1.0
    if index and not middle and not ring and not pinky:
        return G("point"), 1.0
    if not index and not middle and not ring and not pinky:
        # thumb up and thumb tip is the highest point of the hand (in raw image)
        raw = np.asarray(landmarks)
```

```
        if thumb and raw[4, 1] < raw[[8, 12, 16, 20], 1].min():
            return G("thumbs_up"), 1.0
        return G("fist"), 1.0
    return G("idle_palm"), 1.0
```

run_all.py

```
"""
run_all.py
----------
Runs EVERY automatic experiment, then draws the figures.
Run it after all members have recorded data (collect_data.py) and,
optionally, after recording app logs (app.py --log).

    python run_all.py            # everything
    python run_all.py --quick    # skip the slow parts (CNN, leave-one-user-out)

Order matters:
  1. leave-one-user-out          (train.py --loso)
  2. image-CNN baseline           (train_cnn.py)
  3. personalization per user     (personalize.py --user X)
  4. log analysis                 (experiments/analyze_logs.py logs/*.csv)
  5. final model + ablation       (train.py)  -> models/gesture_mlp.pt used by the app
  6. figures                      (make_figures.py)
"""
import argparse
import csv
import glob
import os
import subprocess
import sys
import time

import config

PY = sys.executable


def run(args, required=True):
    print("\n" + "=" * 70 + "\n>>> " + " ".join(args) + "\n" + "=" * 70, flush=True)
    t = time.time()
    r = subprocess.run([PY] + args, cwd=config.ROOT_DIR)
    print(f"<<< finished in {time.time() - t:.0f} s (exit code {r.returncode})")
    if r.returncode != 0 and required:
        raise SystemExit(f"Step failed: {' '.join(args)}")
    return r.returncode == 0


def users_in_data():
    if not os.path.exists(config.LANDMARK_CSV):
        raise SystemExit("No data yet. Every member must run: python collect_data.py --user <name>")
    with open(config.LANDMARK_CSV, newline="") as f:
        return sorted({row["user"] for row in csv.DictReader(f)})


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--quick", action="store_true")
    a = ap.parse_args()

    users = users_in_data()
    print("Users in dataset:", users)
    ok = {}
    if len(users) >= 2 and not a.quick:
        ok["loso"] = run(["train.py", "--loso"], required=False)
    if not a.quick:
        ok["cnn"] = run(["train_cnn.py"], required=False)
    if len(users) >= 2:
        for u in users:
            ok[f"personalize_{u}"] = run(["personalize.py", "--user", u], required=False)
    logs = sorted(glob.glob(os.path.join(config.LOG_DIR, "*.csv")))
    if logs:
        ok["logs"] = run([os.path.join("experiments", "analyze_logs.py")] + logs, required=False)
    else:
        print("\n(no logs in logs/ - record them with:  python app.py --log --dry-run)")
    ok["train"] = run(["train.py"])            # last: this is the model the app uses
    ok["figures"] = run(["make_figures.py"], required=False)

```

```
    print("\nSUMMARY")
    for k, v in ok.items():
        print(f"  {k:<25} {'OK' if v else 'FAILED (see messages above)'}")
    print("\nResults : results/   Figures : results/figures/")


if __name__ == "__main__":
    main()
```

train.py

```
"""
train.py
--------
Train the gesture MLP and run all classification experiments.

    python train.py                     # random 80/20 split, ablation + baselines
    python train.py --test-user guna    # train on the other users, test on 'guna'
    python train.py --loso              # leave-one-user-out over every user

Outputs (in results/):
    ablation.md / ablation.csv     table: feature mode, augmentation, baselines
    loso.md                        table: accuracy on unseen users (with --loso)
    confusion_matrix.png           confusion matrix of the final MLP
    training_curve.png             loss / accuracy per epoch
    classification_report.txt      precision / recall / F1 per gesture
    metrics.json                   all numbers in one file
and the deployed model in models/gesture_mlp.pt
"""
import argparse
import csv
import json
import os
import time

import numpy as np
import torch
import torch.nn as nn
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, classification_report
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier

import config
from features import extract_features, augment_landmarks, FEATURE_MODES
from model import GestureMLP, save_model, count_params
from rules import classify_rules


# ---------------------------------------------------------------------------
# Data
# ---------------------------------------------------------------------------
def load_dataset(path=config.LANDMARK_CSV):
    """Returns users (N,), labels (N,), handedness (N,), landmarks (N,21,3)."""
    users, labels, hands, lms = [], [], [], []
    with open(path, newline="") as f:
        for row in csv.DictReader(f):
            if row["label"] not in config.GESTURES:
                continue
            users.append(row["user"])
            labels.append(config.GESTURES.index(row["label"]))
            hands.append(row["handedness"])
            lms.append([float(row[f"{a}{i}"]) for i in range(21) for a in "xyz"])
    if not labels:
        raise SystemExit(f"No data in {path}. Run collect_data.py first.")
    return (np.array(users), np.array(labels), np.array(hands),
            np.array(lms, dtype=np.float32).reshape(-1, 21, 3))


def featurize(lms, hands, mode):
    return np.stack([extract_features(l, h, mode) for l, h in zip(lms, hands)]).astype(np.float32)


def augment(lms, hands, labels, copies, seed=config.SEED):
```

```
    """Add `copies` randomly-augmented versions of every training sample."""
    if copies <= 0:
        return lms, hands, labels
    rng = np.random.default_rng(seed)
    aug = [augment_landmarks(l, rng) for _ in range(copies) for l in lms]
    return (np.concatenate([lms, np.stack(aug)]),
            np.concatenate([hands] + [hands] * copies),
            np.concatenate([labels] + [labels] * copies))


# ---------------------------------------------------------------------------
# MLP training
# ---------------------------------------------------------------------------
def train_mlp(Xtr, ytr, Xval, yval, epochs=config.EPOCHS, verbose=True, seed=config.SEED):
    """Train on standardized features. Keeps the weights of the best validation epoch."""
    torch.manual_seed(seed)
    model = GestureMLP(Xtr.shape[1], config.NUM_CLASSES)
    opt = torch.optim.AdamW(model.parameters(), lr=config.LEARNING_RATE, weight_decay=1e-4)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=epochs)
    loss_fn = nn.CrossEntropyLoss(label_smoothing=0.05)
    Xtr_t, ytr_t = torch.from_numpy(Xtr), torch.from_numpy(ytr).long()
    Xval_t, yval_t = torch.from_numpy(Xval), torch.from_numpy(yval).long()

    hist = {"train_loss": [], "val_loss": [], "val_acc": []}
    best_acc, best_state = -1, None
    for ep in range(epochs):
        model.train()
        perm = torch.randperm(len(Xtr_t))
        tot = 0.0
        for i in range(0, len(perm), config.BATCH_SIZE):
            idx = perm[i:i + config.BATCH_SIZE]
            if len(idx) < 2:          # BatchNorm needs >1 sample
                continue
            opt.zero_grad()
            loss = loss_fn(model(Xtr_t[idx]), ytr_t[idx])
            loss.backward()
            opt.step()
            tot += loss.item() * len(idx)
        sched.step()

        model.eval()
        with torch.no_grad():
            out = model(Xval_t)
            vloss = loss_fn(out, yval_t).item()
            vacc = (out.argmax(1) == yval_t).float().mean().item()
        hist["train_loss"].append(tot / len(Xtr_t))
        hist["val_loss"].append(vloss)
        hist["val_acc"].append(vacc)
        if vacc > best_acc:
            best_acc = vacc
            best_state = {k: v.clone() for k, v in model.state_dict().items()}
        if verbose and (ep % 10 == 0 or ep == epochs - 1):
            print(f"  epoch {ep:3d}  train_loss {hist['train_loss'][-1]:.4f}  "
                  f"val_loss {vloss:.4f}  val_acc {vacc:.4f}")
    model.load_state_dict(best_state)
    model.eval()
    return model, hist


@torch.no_grad()
def mlp_predict(model, X):
    return model(torch.from_numpy(X)).argmax(1).numpy()


def time_per_sample(fn, n=300):
    """Average time (ms) for one call of fn() - single-sample latency as in the live app."""
    fn()  # warm-up
    t0 = time.perf_counter()
    for _ in range(n):
        fn()
    return (time.perf_counter() - t0) / n * 1000


def run_mlp_experiment(lm_tr, h_tr, y_tr, lm_te, h_te, y_te, mode, aug_copies, verbose=False):
    """Full pipeline for one MLP setting. Returns (model, mean, std, metrics, history)."""
    # validation split (10 %) taken from the training users only
    idx_tr, idx_val = train_test_split(np.arange(len(y_tr)), test_size=0.1,
                                       random_state=config.SEED, stratify=y_tr)
    a_lm, a_h, a_y = augment(lm_tr[idx_tr], h_tr[idx_tr], y_tr[idx_tr], aug_copies)
```

```
    Xtr = featurize(a_lm, a_h, mode)
    mean, std = Xtr.mean(0), Xtr.std(0) + 1e-6
    norm = lambda X: ((X - mean) / std).astype(np.float32)
    Xval = norm(featurize(lm_tr[idx_val], h_tr[idx_val], mode))
    Xte = norm(featurize(lm_te, h_te, mode))

    model, hist = train_mlp(norm(Xtr), a_y, Xval, y_tr[idx_val], verbose=verbose)
    pred = mlp_predict(model, Xte)

    # latency: feature extraction + forward pass for ONE sample
    torch.set_num_threads(1)
    l0, h0 = lm_te[0], h_te[0]
    def one():
        f = norm(extract_features(l0, h0, mode)[None])
        with torch.no_grad():
            model(torch.from_numpy(f))
    ms = time_per_sample(one)
    torch.set_num_threads(os.cpu_count() or 1)

    m = {"acc": accuracy_score(y_te, pred), "macro_f1": f1_score(y_te, pred, average="macro"),
         "latency_ms": ms, "params": count_params(model)}
    return model, mean, std, m, hist, pred


def run_sklearn(clf, lm_tr, h_tr, y_tr, lm_te, h_te, y_te):
    Xtr, Xte = featurize(lm_tr, h_tr, "full"), featurize(lm_te, h_te, "full")
    mean, std = Xtr.mean(0), Xtr.std(0) + 1e-6
    clf.fit((Xtr - mean) / std, y_tr)
    pred = clf.predict((Xte - mean) / std)
    x0 = ((extract_features(lm_te[0], h_te[0], "full") - mean) / std)[None]
    ms = time_per_sample(lambda: clf.predict(x0), n=100)
    return {"acc": accuracy_score(y_te, pred), "macro_f1": f1_score(y_te, pred, average="macro"),
            "latency_ms": ms}


def run_rules(lm_te, h_te, y_te):
    pred = np.array([classify_rules(l, h)[0] for l, h in zip(lm_te, h_te)])
    ms = time_per_sample(lambda: classify_rules(lm_te[0], h_te[0]))
    return {"acc": accuracy_score(y_te, pred), "macro_f1": f1_score(y_te, pred, average="macro"),
            "latency_ms": ms}


# ---------------------------------------------------------------------------
# Plots and tables
# ---------------------------------------------------------------------------
def plot_confusion(y_true, y_pred, path, title):
    cm = confusion_matrix(y_true, y_pred, labels=range(config.NUM_CLASSES))
    cmn = cm / np.maximum(cm.sum(1, keepdims=True), 1)
    fig, ax = plt.subplots(figsize=(6.5, 5.5))
    im = ax.imshow(cmn, cmap="Blues", vmin=0, vmax=1)
    ax.set_xticks(range(config.NUM_CLASSES)); ax.set_yticks(range(config.NUM_CLASSES))
    ax.set_xticklabels(config.GESTURES, rotation=45, ha="right"); ax.set_yticklabels(config.GESTURES)
    for i in range(config.NUM_CLASSES):
        for j in range(config.NUM_CLASSES):
            ax.text(j, i, f"{cmn[i, j]*100:.0f}", ha="center", va="center",
                    color="white" if cmn[i, j] > 0.5 else "black", fontsize=8)
    ax.set_xlabel("Predicted"); ax.set_ylabel("True"); ax.set_title(title)
    fig.colorbar(im, ax=ax, fraction=0.046)
    fig.tight_layout(); fig.savefig(path, dpi=200); plt.close(fig)


def plot_history(hist, path):
    fig, ax = plt.subplots(1, 2, figsize=(9, 3.5))
    ax[0].plot(hist["train_loss"], label="train"); ax[0].plot(hist["val_loss"], label="val")
    ax[0].set_xlabel("epoch"); ax[0].set_ylabel("loss"); ax[0].legend()
    ax[1].plot(hist["val_acc"]); ax[1].set_xlabel("epoch"); ax[1].set_ylabel("val accuracy")
    fig.tight_layout(); fig.savefig(path, dpi=200); plt.close(fig)


def write_table(rows, path_md, path_csv, cols):
    with open(path_csv, "w", newline="") as f:
        w = csv.writer(f); w.writerow(cols)
        for r in rows:
            w.writerow([r.get(c, "") for c in cols])
    with open(path_md, "w") as f:
        f.write("| " + " | ".join(cols) + " |\n|" + "---|" * len(cols) + "\n")
        for r in rows:
            cells = []
            for c in cols:
```

```
                v = r.get(c, "")
                cells.append(f"{v*100:.2f}" if c in ("acc", "macro_f1") and v != "" else
                             f"{v:.3f}" if isinstance(v, float) else str(v))
            f.write("| " + " | ".join(cells) + " |\n")


# ---------------------------------------------------------------------------
# Main
# ---------------------------------------------------------------------------
def split(users, labels, test_user=None):
    if test_user:
        te = users == test_user
        if te.sum() == 0:
            raise SystemExit(f"No samples for user {test_user}")
        return np.where(~te)[0], np.where(te)[0]
    return train_test_split(np.arange(len(labels)), test_size=0.2,
                            random_state=config.SEED, stratify=labels)


def ablation(users, labels, hands, lms, test_user=None):
    tr, te = split(users, labels, test_user)
    args = (lms[tr], hands[tr], labels[tr], lms[te], hands[te], labels[te])
    print(f"Train samples: {len(tr)}   Test samples: {len(te)}   "
          f"(test = {'user ' + test_user if test_user else 'random 20%'})")
    rows, results = [], {}

    print("\n[1] Rule-based baseline (finger-up heuristics)")
    r = run_rules(lms[te], hands[te], labels[te]); r["method"] = "Rule-based (fingers up)"
    rows.append(r)

    for name, clf in [("SVM (RBF)", SVC(C=10, gamma="scale")),
                      ("Random Forest", RandomForestClassifier(200, random_state=config.SEED)),
                      ("kNN (k=5)", KNeighborsClassifier(5))]:
        print(f"[2] {name} on full features")
        r = run_sklearn(clf, *args); r["method"] = f"{name}, full features"; rows.append(r)

    final = None
    for mode in FEATURE_MODES:
        for aug in ([0, config.AUGMENT_COPIES] if mode == "full" else [config.AUGMENT_COPIES]):
            print(f"[3] MLP  features={mode}  augmentation copies={aug}")
            model, mean, std, m, hist, pred = run_mlp_experiment(*args, mode, aug,
                                                                 verbose=(mode == "full" and aug > 0))
            m["method"] = f"MLP, {mode} features" + ("" if aug else ", no augmentation")
            rows.append(m)
            if mode == "full" and aug > 0:
                final = (model, mean, std, hist, pred, m)

    for r in rows:
        print(f"  {r['method']:<40} acc={r['acc']*100:6.2f}%  F1={r['macro_f1']*100:6.2f}%  "
              f"latency={r['latency_ms']:.3f} ms")
    cols = ["method", "acc", "macro_f1", "latency_ms", "params"]
    tag = f"_{test_user}" if test_user else ""
    write_table(rows, os.path.join(config.RESULTS_DIR, f"ablation{tag}.md"),
                os.path.join(config.RESULTS_DIR, f"ablation{tag}.csv"), cols)

    model, mean, std, hist, pred, m = final
    save_model(model, config.GESTURE_MODEL_PATH, "full", mean, std,
               extra={"test_acc": m["acc"], "test_user": test_user})
    plot_confusion(labels[te], pred, os.path.join(config.RESULTS_DIR, f"confusion_matrix{tag}.png"),
                   f"MLP (full features) - acc {m['acc']*100:.1f}%")
    plot_history(hist, os.path.join(config.RESULTS_DIR, f"training_curve{tag}.png"))
    with open(os.path.join(config.RESULTS_DIR, f"classification_report{tag}.txt"), "w") as f:
        f.write(classification_report(labels[te], pred, labels=range(config.NUM_CLASSES),
                                      target_names=config.GESTURES, digits=4, zero_division=0))
    results["rows"] = rows
    print(f"\nSaved model to {config.GESTURE_MODEL_PATH}")
    return results


def loso(users, labels, hands, lms):
    """Leave-one-user-out: how well does it work for a person not in the training data?"""
    rows = []
    for u in sorted(set(users)):
        tr, te = split(users, labels, u)
        args = (lms[tr], hands[tr], labels[tr], lms[te], hands[te], labels[te])
        print(f"\n=== test user: {u}  (train {len(tr)}, test {len(te)}) ===")
        r_rule = run_rules(lms[te], hands[te], labels[te])
        r_svm = run_sklearn(SVC(C=10, gamma="scale"), *args)
        _, _, _, r_mlp, _, _ = run_mlp_experiment(*args, "full", config.AUGMENT_COPIES)
```

```
        _, _, _, r_raw, _, _ = run_mlp_experiment(*args, "raw", 0)
        rows.append({"test_user": str(u), "rules": r_rule["acc"], "svm": r_svm["acc"],
                     "mlp_raw": r_raw["acc"], "mlp_full": r_mlp["acc"]})
        print(rows[-1])
    mean = {k: float(np.mean([r[k] for r in rows])) for k in ("rules", "svm", "mlp_raw", "mlp_full")}
    rows.append({"test_user": "MEAN", **mean})
    with open(os.path.join(config.RESULTS_DIR, "loso.md"), "w") as f:
        f.write("| Test user | Rule-based | SVM (full) | MLP (raw, no aug) | MLP (full + aug, ours) |\n"
                "|---|---|---|---|---|\n")
        for r in rows:
            f.write(f"| {r['test_user']} | {r['rules']*100:.2f} | {r['svm']*100:.2f} | "
                    f"{r['mlp_raw']*100:.2f} | {r['mlp_full']*100:.2f} |\n")
    print(open(os.path.join(config.RESULTS_DIR, "loso.md")).read())
    return rows


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--test-user", default=None)
    ap.add_argument("--loso", action="store_true")
    args = ap.parse_args()

    users, labels, hands, lms = load_dataset()
    print(f"Loaded {len(labels)} samples from users {sorted(set(users))}")
    print("Per class:", {g: int((labels == i).sum()) for i, g in enumerate(config.GESTURES)})

    out = {"n_samples": int(len(labels)), "users": sorted(set(users.tolist()))}
    if args.loso:
        out["loso"] = loso(users, labels, hands, lms)
    else:
        out["ablation"] = ablation(users, labels, hands, lms, args.test_user)["rows"]
    name = "metrics_loso.json" if args.loso else f"metrics{'_' + args.test_user if args.test_user else ''}.json"
    with open(os.path.join(config.RESULTS_DIR, name), "w") as f:
        json.dump(out, f, indent=2, default=float)


if __name__ == "__main__":
    main()
```

train_cnn.py

```
"""
train_cnn.py
------------
Baseline: a small CNN that classifies the gesture directly from the hand
IMAGE crop (saved by collect_data.py in data/images/<label>/<user>_*.jpg).

We compare it with our keypoint MLP on accuracy, speed and model size,
to justify the choice of keypoint features. (Both still need MediaPipe to
find and crop the hand, so the comparison is only about the classifier.)

    python train_cnn.py                    # random 80/20 split
    python train_cnn.py --test-user guna   # test on an unseen person
Output: results/cnn_baseline.md
"""
import argparse
import glob
import os
import time

import cv2
import numpy as np
import torch
import torch.nn as nn
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score

import config

IMG = 64


class SmallCNN(nn.Module):
    def __init__(self, n):
        super().__init__()
        def block(i, o):
            return nn.Sequential(nn.Conv2d(i, o, 3, padding=1), nn.BatchNorm2d(o), nn.ReLU(), nn.MaxPool2d(2))
        self.f = nn.Sequential(block(3, 32), block(32, 64), block(64, 128), block(128, 128))
```

```
        self.head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Dropout(0.3), nn.Linear(128, n))

    def forward(self, x):
        return self.head(self.f(x))


def load_images():
    X, y, users = [], [], []
    for ci, g in enumerate(config.GESTURES):
        for p in glob.glob(os.path.join(config.IMAGE_DIR, g, "*.jpg")):
            im = cv2.imread(p)
            if im is None:
                continue
            X.append(cv2.resize(im, (IMG, IMG))[:, :, ::-1])
            y.append(ci)
            users.append(os.path.basename(p).rsplit("_", 1)[0])
    if not X:
        raise SystemExit("No images found. Run collect_data.py (with --save-images 1) first.")
    X = np.stack(X).astype(np.float32) / 255.0
    return X.transpose(0, 3, 1, 2), np.array(y), np.array(users)


def augment_batch(x):
    """Random brightness / contrast and small shifts (training only)."""
    b = x.shape[0]
    x = x * torch.empty(b, 1, 1, 1).uniform_(0.7, 1.3) + torch.empty(b, 1, 1, 1).uniform_(-0.1, 0.1)
    dx, dy = np.random.randint(-4, 5, size=2)
    x = torch.roll(x, shifts=(int(dy), int(dx)), dims=(2, 3))
    return x.clamp(0, 1)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--test-user", default=None)
    ap.add_argument("--epochs", type=int, default=25)
    args = ap.parse_args()
    torch.manual_seed(config.SEED); np.random.seed(config.SEED)

    X, y, users = load_images()
    print(f"{len(y)} images, users {sorted(set(users))}")
    if args.test_user:
        tr, te = np.where(users != args.test_user)[0], np.where(users == args.test_user)[0]
    else:
        tr, te = train_test_split(np.arange(len(y)), test_size=0.2, random_state=config.SEED, stratify=y)

    model = SmallCNN(config.NUM_CLASSES)
    opt = torch.optim.AdamW(model.parameters(), lr=2e-3, weight_decay=1e-4)
    loss_fn = nn.CrossEntropyLoss(label_smoothing=0.05)
    Xtr, ytr = torch.from_numpy(X[tr]), torch.from_numpy(y[tr]).long()
    Xte = torch.from_numpy(X[te])
    t_start = time.time()
    for ep in range(args.epochs):
        model.train()
        perm = torch.randperm(len(Xtr))
        tot = 0
        for i in range(0, len(perm), 64):
            idx = perm[i:i + 64]
            if len(idx) < 2:
                continue
            opt.zero_grad()
            loss = loss_fn(model(augment_batch(Xtr[idx])), ytr[idx])
            loss.backward(); opt.step()
            tot += loss.item() * len(idx)
        print(f"epoch {ep:2d} loss {tot/len(Xtr):.4f}")
    train_time = time.time() - t_start

    model.eval()
    with torch.no_grad():
        pred = torch.cat([model(Xte[i:i + 256]).argmax(1) for i in range(0, len(Xte), 256)]).numpy()
    acc, f1 = accuracy_score(y[te], pred), f1_score(y[te], pred, average="macro")

    torch.set_num_threads(1)
    x0 = Xte[:1]
    with torch.no_grad():
        model(x0)
        t0 = time.perf_counter()
        for _ in range(100):
            model(x0)
    ms = (time.perf_counter() - t0) / 100 * 1000
```

```
    params = sum(p.numel() for p in model.parameters())

    tag = f"_{args.test_user}" if args.test_user else ""
    out = os.path.join(config.RESULTS_DIR, f"cnn_baseline{tag}.md")
    with open(out, "w") as f:
        f.write("| Method | Accuracy (%) | Macro F1 (%) | Latency per sample (ms) | Parameters | Train time (s) |\n"
                "|---|---|---|---|---|---|\n")
        f.write(f"| CNN on {IMG}x{IMG} hand image | {acc*100:.2f} | {f1*100:.2f} | {ms:.2f} | {params} | {train_time
:.0f} |\n")
    print(open(out).read())
    torch.save(model.state_dict(), os.path.join(config.MODEL_DIR, "gesture_cnn.pt"))


if __name__ == "__main__":
    main()
```

virtual_keyboard.py

```
"""
virtual_keyboard.py
-------------------
On-screen "air keyboard" drawn on the camera view.
  * point with the index finger  -> the key under the fingertip is highlighted
  * pinch (thumb + index)        -> that key is typed

"Hover lock": when you pinch, your fingertip moves a little, so the key under
it may change. We therefore type the key that was hovered just BEFORE the
pinch started (the last key seen while the gesture was "point").
"""
import time

import cv2

ROWS = [
    list("1234567890"),
    list("QWERTYUIOP"),
    list("ASDFGHJKL"),
    list("ZXCVBNM"),
    ["SPACE", "BACK", "ENTER"],
]
SPECIAL = {"SPACE": "space", "BACK": "backspace", "ENTER": "enter"}


class VirtualKeyboard:
    def __init__(self, frame_w, frame_h, top_ratio=0.22, bottom_ratio=0.75, width_ratio=0.80):
        # keyboard sits between 22% and 75% of the frame height, so the bottom
        # row (SPACE / BACK / ENTER) is easy to reach without leaving the camera view
        self.keys = []            # list of (label, x1, y1, x2, y2)
        top = int(frame_h * top_ratio)
        bottom = int(frame_h * bottom_ratio)
        rows = len(ROWS)
        key_h = (bottom - top) // rows
        side = int(frame_w * (1 - width_ratio) / 2)   # empty space left and right
        key_w = (frame_w - 2 * side) // 10
        for r, row in enumerate(ROWS):
            y1 = top + r * key_h
            if r == rows - 1:     # bottom row: wide keys
                widths = [5, 2.5, 2.5]
                x = side
                for lab, wf in zip(row, widths):
                    w = int(key_w * wf)
                    self.keys.append((lab, x + 3, y1 + 3, x + w - 3, y1 + key_h - 3))
                    x += w
            else:
                offset = side + (10 - len(row)) * key_w // 2
                for c, lab in enumerate(row):
                    x1 = offset + c * key_w
                    self.keys.append((lab, x1 + 3, y1 + 3, x1 + key_w - 3, y1 + key_h - 3))
        self.hover = None
        self.locked = None        # key hovered just before a pinch
        self.flash_key, self.flash_t = None, 0.0
        self.typed = ""

    def key_at(self, x, y):
        for lab, x1, y1, x2, y2 in self.keys:
            if x1 <= x <= x2 and y1 <= y <= y2:
                return lab
        return None
```

```
    def update_hover(self, x, y, is_pointing):
        """Call every frame with the fingertip pixel position."""
        self.hover = self.key_at(x, y)
        if is_pointing:
            self.locked = self.hover

    def press(self):
        """Called on pinch onset. Returns the key name for the controller, or None."""
        lab = self.locked or self.hover
        if lab is None:
            return None
        self.flash_key, self.flash_t = lab, time.time()
        if lab == "BACK":
            self.typed = self.typed[:-1]
        elif lab == "SPACE":
            self.typed += " "
        elif lab == "ENTER":
            self.typed = ""
        else:
            self.typed += lab.lower()
        self.typed = self.typed[-40:]
        return SPECIAL.get(lab, lab)

    def draw(self, frame):
        overlay = frame.copy()
        for lab, x1, y1, x2, y2 in self.keys:
            if lab == self.flash_key and time.time() - self.flash_t < 0.25:
                col = (0, 200, 0)
            elif lab == self.hover:
                col = (0, 180, 255)
            else:
                col = (60, 60, 60)
            cv2.rectangle(overlay, (x1, y1), (x2, y2), col, -1)
        cv2.addWeighted(overlay, 0.55, frame, 0.45, 0, frame)
        for lab, x1, y1, x2, y2 in self.keys:
            cv2.rectangle(frame, (x1, y1), (x2, y2), (230, 230, 230), 1)
            fs = 0.7 if len(lab) == 1 else 0.55
            (tw, th), _ = cv2.getTextSize(lab, cv2.FONT_HERSHEY_SIMPLEX, fs, 2)
            cv2.putText(frame, lab, ((x1 + x2 - tw) // 2, (y1 + y2 + th) // 2),
                        cv2.FONT_HERSHEY_SIMPLEX, fs, (255, 255, 255), 2, cv2.LINE_AA)
        # typed-text box
        top = self.keys[0][2] - 45
        cv2.rectangle(frame, (20, top), (frame.shape[1] - 20, top + 35), (255, 255, 255), -1)
        cv2.putText(frame, self.typed + "|", (28, top + 25), cv2.FONT_HERSHEY_SIMPLEX,
                    0.75, (0, 0, 0), 2, cv2.LINE_AA)
```

voting.py

```
"""
voting.py
---------
Confidence + temporal voting ("debouncing") of the per-frame gesture predictions.

The classifier sometimes flips to a wrong class for one or two frames
(for example while the hand is moving from "point" to "pinch").
If we acted on every frame, these flips would create unwanted clicks.

Rule: the stable gesture only changes to class c when, among the last
VOTE_WINDOW frames, c was predicted at least VOTE_MIN_COUNT times AND the
mean confidence of those predictions is above CONF_THRESHOLD.
Otherwise the previous stable gesture is kept.
"""
from collections import deque, Counter

import config


class GestureVoter:
    def __init__(self, window=config.VOTE_WINDOW, min_count=config.VOTE_MIN_COUNT,
                 conf_thr=config.CONF_THRESHOLD, enabled=True):
        self.window, self.min_count, self.conf_thr = window, min_count, conf_thr
        self.enabled = enabled
        self.buf = deque(maxlen=window)
        self.stable = config.GESTURES.index("idle_palm")

    def reset(self):
        self.buf.clear()
```

```
        self.stable = config.GESTURES.index("idle_palm")

    def update(self, label, conf):
        """Feed one frame's prediction; returns the stable label."""
        if not self.enabled:
            self.stable = label
            return label
        self.buf.append((label, conf))
        counts = Counter(l for l, _ in self.buf)
        best, n = counts.most_common(1)[0]
        mean_conf = sum(c for l, c in self.buf if l == best) / n
        if n >= self.min_count and mean_conf >= self.conf_thr:
            self.stable = best
        return self.stable
```

experiments/analyze_logs.py

```
"""
experiments/analyze_logs.py
---------------------------
Offline analysis of the per-frame logs recorded with  `python app.py --log`.
Because the log keeps the RAW hand positions and RAW per-frame predictions,
we can replay the same session through every filter / voting setting and
compare them fairly on identical input.

    python experiments/analyze_logs.py logs/session_XXXX.csv [more logs ...]

Suggested recording (about 1 minute each, app in --dry-run is fine):
  1. "still"  : point at the screen and hold the hand as still as you can (jitter)
  2. "move"   : move the cursor around in big and small motions           (lag)
  3. "clicks" : point, pinch, point, pinch ... EXACTLY 30 times            (anchor drift, voting)
                -> in voting_comparison.md the best setting detects close to 30 clicks

Outputs in results/:
  performance.md        FPS and processing time per frame
  filter_comparison.md  jitter (px) and lag (ms) for none / moving_avg / kalman / one_euro
  filter_trace.png      raw vs filtered cursor trace
  voting_comparison.md  label flicker and false gesture onsets with / without voting
  anchor_drift.md       how far the cursor moves at the moment of a click, per anchor point
"""
import argparse
import csv
import os
import sys

import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
import config                                   # noqa: E402
from filters import make_filter, FILTERS        # noqa: E402
from voting import GestureVoter                 # noqa: E402
from app import to_screen                       # noqa: E402

SCREEN_W, SCREEN_H = 1920, 1080


def read_log(path):
    with open(path, newline="") as f:
        rows = list(csv.DictReader(f))
    return rows


def runs_with_hand(rows):
    """Split the log into continuous pieces where a hand was visible."""
    runs, cur = [], []
    for r in rows:
        if r["hand"] == "1":
            cur.append(r)
        elif cur:
            runs.append(cur); cur = []
    if cur:
        runs.append(cur)
    return [r for r in runs if len(r) >= 15]
```

```python
def gaussian_smooth(x, sigma=2.0):
    """Zero-phase (centered) smoothing -> used as the 'intended' hand path (reference)."""
    k = int(3 * sigma)
    w = np.exp(-0.5 * (np.arange(-k, k + 1) / sigma) ** 2); w /= w.sum()
    xp = np.pad(x, ((k, k), (0, 0)), mode="edge")
    return np.stack([np.convolve(xp[:, d], w, mode="valid") for d in range(x.shape[1])], 1)


# ---------------------------------------------------------------------
def performance(all_rows, out):
    fps = np.array([float(r["fps"]) for r in all_rows if float(r["fps"]) > 0])
    ms = np.array([float(r["proc_ms"]) for r in all_rows])
    with open(out, "w") as f:
        f.write("| Metric | Value |\n|---|---|\n")
        f.write(f"| Frames analysed | {len(all_rows)} |\n")
        f.write(f"| Mean FPS | {fps.mean():.1f} |\n")
        f.write(f"| Processing time per frame, mean (ms) | {ms.mean():.1f} |\n")
        f.write(f"| Processing time per frame, 95th percentile (ms) | {np.percentile(ms, 95):.1f} |\n")
    print(open(out).read())


def filter_comparison(runs, out_md, out_png):
    res = {n: {"jit": [], "err": [], "lag": []} for n in FILTERS}
    trace_done = False
    for run in runs:
        t = np.array([float(r["t"]) for r in run])
        raw = np.array([to_screen(float(r["mcp_x"]), float(r["mcp_y"]), SCREEN_W, SCREEN_H)
                        for r in run])
        ref = gaussian_smooth(raw)
        dtm = np.median(np.diff(t)) if len(t) > 1 else 1 / 30
        speed = np.r_[0, np.linalg.norm(np.diff(ref, axis=0), axis=1) / np.maximum(np.diff(t), 1e-3)]
        still, moving = speed < 150, speed > 400            # px/s

        outs = {}
        for name in FILTERS:
            flt = make_filter(name)
            y = np.array([flt(x[0], x[1], ti) for x, ti in zip(raw, t)])
            outs[name] = y
            step = np.r_[0, np.linalg.norm(np.diff(y, axis=0), axis=1)]
            if still.sum() > 5:
                res[name]["jit"].append(step[still].mean())      # px moved per frame when hand still
            if moving.sum() > 5:
                res[name]["err"].append(np.linalg.norm(y - ref, axis=1)[moving].mean())
                # lag = shift (in frames) that best aligns filtered signal with the reference
                best = min(range(0, 15), key=lambda s: np.mean(np.linalg.norm(
                    y[s:] - ref[:len(ref) - s], axis=1)))
                res[name]["lag"].append(best * dtm * 1000)

        if not trace_done and len(run) > 90:
            n = min(len(run), 240)
            fig, ax = plt.subplots(figsize=(8, 3.2))
            ax.plot(t[:n] - t[0], raw[:n, 0], color="0.75", lw=1, label="raw")
            for name in ("moving_avg", "kalman", "one_euro"):
                ax.plot(t[:n] - t[0], outs[name][:n, 0], lw=1.3, label=name)
            ax.set_xlabel("time (s)"); ax.set_ylabel("cursor x (px)"); ax.legend(ncol=4, fontsize=8)
            fig.tight_layout(); fig.savefig(out_png, dpi=200); plt.close(fig)
            trace_done = True

    f = lambda v: f"{np.mean(v):.2f}" if v else "n/a"
    with open(out_md, "w") as fo:
        fo.write("| Filter | Jitter when still (px/frame) | Error when moving (px) | Lag (ms) |\n"
                 "|---|---|---|---|\n")
        for name in FILTERS:
            r = res[name]
            fo.write(f"| {name} | {f(r['jit'])} | {f(r['err'])} | {f(r['lag'])} |\n")
    print(open(out_md).read())


def segments(labels):
    """Run-length encoding: list of (label, length)."""
    segs, prev, n = [], None, 0
    for l in labels:
        if l == prev:
            n += 1
        else:
            if prev is not None:
                segs.append((prev, n))
            prev, n = l, 1
    if prev is not None:
```

```
        segs.append((prev, n))
    return segs


def voting_comparison(runs, out_md):
    settings = [("No voting (raw per-frame)", None), ("Voting N=3, k=2", (3, 2)),
                ("Voting N=5, k=3 (default)", (5, 3)), ("Voting N=7, k=5", (7, 5))]
    SKIP = 7
    total_t = sum(float(r[-1]["t"]) - float(r[SKIP]["t"]) for r in runs) / 60.0  # minutes
    with open(out_md, "w") as fo:
        fo.write("| Setting | Gesture changes / min | Short false segments (<3 frames) / min | "
                 "Left clicks detected (total) |\n|---|---|---|---|\n")
        for name, s in settings:
            changes = short = clicks = 0
            for run in runs:
                lab = [int(r["raw_label"]) for r in run]
                conf = [float(r["conf"]) for r in run]
                if s is None:
                    out = lab
                else:
                    v = GestureVoter(window=s[0], min_count=s[1])
                    out = [v.update(l, c) for l, c in zip(lab, conf)]
                # skip the first frames of every run: the voter starts from "idle"
                # and needs a few frames to fill its window (same cut for all settings)
                seg = segments(out[SKIP:])
                changes += len(seg) - 1
                short += sum(1 for _, n in seg[1:-1] if n < 3)
                clicks += sum(1 for l, _ in seg if l == config.GESTURES.index("pinch_index"))
            fo.write(f"| {name} | {changes/total_t:.1f} | {short/total_t:.1f} | {clicks} |\n")
    print(open(out_md).read())


def anchor_drift(runs, out_md, before=4):
    """Cursor displacement between `before` frames before a pinch onset and the onset itself."""
    pinch = config.GESTURES.index("pinch_index")
    d = {"index_tip": [], "index_mcp": [], "palm": []}
    cols = {"index_tip": ("tip_x", "tip_y"), "index_mcp": ("mcp_x", "mcp_y"), "palm": ("palm_x", "palm_y")}
    for run in runs:
        st = [int(r["stable_label"]) for r in run]
        for i in range(before, len(run)):
            if st[i] == pinch and st[i - 1] != pinch:
                for a, (cx, cy) in cols.items():
                    p1 = to_screen(float(run[i - before][cx]), float(run[i - before][cy]), SCREEN_W, SCREEN_H)
                    p2 = to_screen(float(run[i][cx]), float(run[i][cy]), SCREEN_W, SCREEN_H)
                    d[a].append(np.hypot(p2[0] - p1[0], p2[1] - p1[1]))
    with open(out_md, "w") as fo:
        fo.write(f"| Anchor point | Cursor drift during pinch (px), mean | median | clicks |\n|---|---|---|---|\n")
        for a, v in d.items():
            if v:
                fo.write(f"| {a} | {np.mean(v):.1f} | {np.median(v):.1f} | {len(v)} |\n")
            else:
                fo.write(f"| {a} | n/a | n/a | 0 |\n")
    print(open(out_md).read())


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("logs", nargs="+")
    a = ap.parse_args()
    all_rows, runs = [], []
    for p in a.logs:
        rows = read_log(p)
        all_rows += rows
        runs += runs_with_hand(rows)
    if not runs:
        raise SystemExit("No usable hand segments in the logs.")
    R = config.RESULTS_DIR
    print("== Performance =="); performance(all_rows, os.path.join(R, "performance.md"))
    print("== Filters ==");     filter_comparison(runs, os.path.join(R, "filter_comparison.md"),
                                                  os.path.join(R, "filter_trace.png"))
    print("== Voting ==");      voting_comparison(runs, os.path.join(R, "voting_comparison.md"))
    print("== Anchor drift =="); anchor_drift(runs, os.path.join(R, "anchor_drift.md"))


if __name__ == "__main__":
    main()
```

experiments/click_test.py

```python
"""
experiments/click_test.py
-------------------------
Target-clicking test (a simple Fitts' law task) to measure how usable the
gesture mouse is, compared with a normal mouse / touchpad.

    python experiments/click_test.py --method mouse   --user guna
    python experiments/click_test.py --method gesture --user guna   (run app.py at the same time)

A red circle appears at a random place. Click it as fast as you can.
After every click (hit or miss) a new circle appears.
Results: results/click_test_<method>_<user>.csv and a summary line in
results/click_test_summary.md
  * movement time (s), error rate (%), throughput (bits/s) = ID / time,
    where ID = log2(distance / width + 1)  (Fitts' index of difficulty)
"""
import argparse
import csv
import math
import os
import random
import sys
import time
import tkinter as tk

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
import config  # noqa: E402


class ClickTest:
    def __init__(self, root, args):
        self.root, self.args = root, args
        self.W, self.H = 1100, 700
        self.c = tk.Canvas(root, width=self.W, height=self.H, bg="white")
        self.c.pack()
        self.c.bind("<Button-1>", self.on_click)
        self.info = self.c.create_text(self.W // 2, 20, text="", font=("Arial", 14))
        self.trial, self.results = 0, []
        self.prev = (self.W // 2, self.H // 2)
        self.next_target()

    def next_target(self):
        r = random.choice(self.args.radii)
        self.r = r
        self.x = random.randint(r + 10, self.W - r - 10)
        self.y = random.randint(r + 40, self.H - r - 10)
        self.c.delete("target")
        self.c.create_oval(self.x - r, self.y - r, self.x + r, self.y + r, fill="red", tags="target")
        self.t0 = time.perf_counter()
        self.c.itemconfig(self.info, text=f"{self.args.method}: trial {self.trial + 1}/{self.args.trials}")

    def on_click(self, e):
        mt = time.perf_counter() - self.t0
        hit = math.hypot(e.x - self.x, e.y - self.y) <= self.r
        dist = math.hypot(self.x - self.prev[0], self.y - self.prev[1])
        idx = math.log2(dist / (2 * self.r) + 1)
        self.results.append({"trial": self.trial, "radius": self.r, "distance": round(dist, 1),
                             "time_s": round(mt, 3), "hit": int(hit), "ID_bits": round(idx, 3)})
        self.prev = (e.x, e.y)
        self.trial += 1
        if self.trial >= self.args.trials:
            self.finish()
        else:
            self.next_target()

    def finish(self):
        R = self.results[1:]            # first trial ignored (start position unknown)
        hits = [r for r in R if r["hit"]]
        mt = sum(r["time_s"] for r in R) / len(R)
        err = 100 * (1 - len(hits) / len(R))
        tp = sum(r["ID_bits"] / r["time_s"] for r in hits) / max(len(hits), 1)
        tag = f"{self.args.method}_{self.args.user}"
        path = os.path.join(config.RESULTS_DIR, f"click_test_{tag}.csv")
        with open(path, "w", newline="") as f:
            w = csv.DictWriter(f, fieldnames=list(R[0].keys())); w.writeheader(); w.writerows(self.results)
        summ = os.path.join(config.RESULTS_DIR, "click_test_summary.md")
        new = not os.path.exists(summ)
        with open(summ, "a") as f:
            if new:
```

```
                f.write("| Method | User | Mean time per target (s) | Error rate (%) | Throughput (bits/s) |\n"
                        "|---|---|---|---|---|\n")
            f.write(f"| {self.args.method} | {self.args.user} | {mt:.2f} | {err:.1f} | {tp:.2f} |\n")
        msg = f"Done!  time {mt:.2f} s   error {err:.1f}%   throughput {tp:.2f} bits/s"
        print(msg)
        self.c.delete("target")
        self.c.itemconfig(self.info, text=msg + "   (close the window)")


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--method", required=True, help="e.g. mouse, touchpad, gesture, gesture_noFilter")
    ap.add_argument("--user", required=True)
    ap.add_argument("--trials", type=int, default=21)
    ap.add_argument("--radii", type=int, nargs="+", default=[20, 35, 50])
    args = ap.parse_args()
    root = tk.Tk(); root.title("Click test")
    ClickTest(root, args)
    root.mainloop()


if __name__ == "__main__":
    main()
```

experiments/typing_test.py

```
"""
experiments/typing_test.py
--------------------------
Typing-speed test for the air keyboard.

    python experiments/typing_test.py --method air_keyboard --user guna

1. Run app.py in another window and switch to KEYBOARD mode (thumbs-up, hold 1 s).
2. Click once inside the text box of this test window (so it receives the keys).
3. Type the phrase shown, then "type" ENTER.
The timer starts at the first character. Results are appended to
results/typing_summary.md : words per minute (WPM = chars/5 per minute) and
character error rate (edit distance / phrase length).
"""
import argparse
import os
import random
import sys
import time
import tkinter as tk

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
import config  # noqa: E402

PHRASES = ["hello world", "image processing", "the quick brown fox", "iit kanpur",
           "gesture control", "deep learning", "open the door", "good morning"]


def edit_distance(a, b):
    d = list(range(len(b) + 1))
    for i in range(1, len(a) + 1):
        prev, d[0] = d[0], i
        for j in range(1, len(b) + 1):
            cur = d[j]
            d[j] = min(d[j] + 1, d[j - 1] + 1, prev + (a[i - 1] != b[j - 1]))
            prev = cur
    return d[len(b)]


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--method", default="air_keyboard")
    ap.add_argument("--user", required=True)
    args = ap.parse_args()
    phrase = random.choice(PHRASES)

    root = tk.Tk(); root.title("Typing test")
    tk.Label(root, text="Type this phrase, then press ENTER:", font=("Arial", 14)).pack(pady=8)
    tk.Label(root, text=phrase, font=("Consolas", 24, "bold")).pack(pady=8)
    entry = tk.Entry(root, font=("Consolas", 22), width=30); entry.pack(pady=8); entry.focus_set()
    result = tk.Label(root, text="", font=("Arial", 14)); result.pack(pady=8)
    state = {"t0": None}
```

```
53
54      def on_key(e):
55          if state["t0"] is None and e.keysym not in ("Return", "BackSpace"):
56              state["t0"] = time.perf_counter()
57
58      def on_enter(e):
59          if state["t0"] is None:
60              return
61          dt = time.perf_counter() - state["t0"]
62          typed = entry.get().strip().lower()
63          wpm = (len(typed) / 5) / (dt / 60)
64          cer = 100 * edit_distance(typed, phrase) / len(phrase)
65          result.config(text=f"time {dt:.1f} s   WPM {wpm:.2f}   char error {cer:.1f}%")
66          summ = os.path.join(config.RESULTS_DIR, "typing_summary.md")
67          new = not os.path.exists(summ)
68          with open(summ, "a") as f:
69              if new:
70                  f.write("| Method | User | Phrase | Time (s) | WPM | Char error (%) |\n|---|---|---|---|---|---|\n")
71              f.write(f"| {args.method} | {args.user} | {phrase} | {dt:.1f} | {wpm:.2f} | {cer:.1f} |\n")
72          print(result.cget("text"))
73
74      entry.bind("<Key>", on_key)
75      entry.bind("<Return>", on_enter)
76      root.mainloop()
77
78
79  if __name__ == "__main__":
80      main()
```